# 高校共享单车系统的布局优化与动态调度机制研究


## 摘要

近年来，共享单车在高校校园内逐渐普及，已经成为学生日常通勤的重要工具，显著提升了出行便利性。然而，其运营过程中也面临着高峰期供需矛盾突出、调度效率低下、停车点布局不合理和故障车辆回收不及时等问题。为提升共享单车的使用效率与服务质量，本文围绕上述问题，基于某高校共享单车运营数据，提出了一套系统的优化方案，旨在提升共享单车系统的运营效率，为高校绿色出行提供科学支持。

**针对问题一**，我们通过分析单车需求与时间、空间等因素的关联性，构建了需求预测函数。然后基于附件提供的部分停车点数据，采用**自然三次样条插值法**构建单车数量分布模型。通过边界条件补充与 MATLAB 插值计算，推算出各停车点在不同时间的单车分布（结果见表 1），并由此估算出校内共享单车**总数下界为 775 辆**，为后续问题提供数据基础。

**针对问题二**，为了优化车辆调度策略，我们首先在需求函数的基础上提出时空需求模型，并综合运力和路径约束，构建了包含多重约束的动态调度模型。随后利用**计算机视觉（OpenCV）**提取道路和停车点，并使用 **Dijkstra 算法**构建停车点间最短路径矩阵，设计以“最大化缓解供需矛盾”和“最小化调度时间成本”为目标的**双目标优化模型**，借助**遗传算法**迭代求解。结果显示，在选定的三个时间段（早高峰前 6:00～7:00、午高峰前 12:30～13:30 和晚高峰前 16:00～17:00）实施调度后，供需矛盾**缓解效率分别达 71.44%、68.96% 和 73.78%**，显著缓解了关键时段供需不均的问题。

**针对问题三**，为评估停车点布局，我们引入了一种综合评估与优化的停车点布局模型。采用带评估指标的 **K-means 聚类算法**生成候选停车点布局方案，随后结合 **TOPSIS 法**，对共计 5 个指标进行量化评估，确定最优聚类数并优化停车点分布。最终选定了停车点数量为 13 时的布局方案。调整后，供需匹配率提升 **18.9 个百分点** (72.4%→91.3%)，高峰期调度效率提高 **17～26 个百分点**，验证了布局优化的有效性。

**针对问题四**，我们聚焦故障单车的回收与维修问题，构建了**容量约束的车辆路径问题模型（CVRP）**，并创新性地设计了融合贪心策略的启发式算法。该模型以最小化故障车辆回收的加权完成时间为目标，通过动态规划方法实现了巡检路径的高效求解。结果显示，优化后的巡检方案仅需两趟即可完成全部回收任务，**总耗时 80.29 分钟**，故障车辆**收集率达 100%**，且两趟运输均满载运行。进一步的路径可视化分析显示，该算法能高效覆盖高需求节点，显著提升了运维系统的响应效率。

综上所述，本文的模型兼具理论严谨性与实际可操作性，能够高效地解决共享单车运营中的关键问题。研究成果不仅为高校共享单车的管理提供了理论依据与实践方案，也为其他共享交通系统的优化提供了可借鉴的思路。未来可进一步引入实时数据与动态调度策略，以适应更为复杂和多变的运营场景，从而为城市共享出行系统、物流配送系统提供理论参考与实践借鉴。



**关键词**：共享单车调度　自然三次样条插值　Dijkstra 最短路　遗传算法　CVRP 模型

# 目录


一、 问题重述 …… 1
1.1 问题背景 …… 1
1.2 问题重述 …… 1
二、 问题分析 …… 1
三、 模型准备 …… 1
3.1 模型假设 …… 1
3.2 符号说明 …… 2
四、 问题一的模型建立与求解 …… 2
4.1 问题一的模型建立 …… 2
4.2 问题一的求解：三次样条插值 …… 3
五、 问题二的模型建立与求解 …… 6
5.1 问题二的模型建立 …… 6
5.1.1 关于时间空间的用车需求模型 …… 6
5.1.2 关于时空的多车辆联合调度模型建立 …… 6
5.1.3 基于遗传算法的模型求解 …… 8
5.2 问题二的求解 …… 9
5.2.1 地图信息的离散化建图过程 …… 9
5.2.2 多源最短路径计算与距离矩阵构建 …… 10
5.2.3 运用遗传模型的问题二结果 …… 11
5.3 问题二模型的评价 …… 14
六、 问题三的模型建立与求解 …… 14
6.1 问题三的模型建立 …… 14
6.1.1 TOPSIS 评价指标的选取 …… 15
6.1.2 带评价指标的 K-means 聚类模型 …… 16
6.2 问题三的求解：使用 TOPSIS 与 K-Means 综合确定最优站点分布方案 …… 17
七、 问题四的模型建立与求解 …… 18
7.1 问题四的模型建立 …… 18
7.1.1 模型构建：考虑加权完成时间的静态 CVRP 模型 …… 18
7.1.2 模型构建：多趟 CVRP 问题的贪心启发式算法 …… 19
7.2 问题四的求解 …… 20
八、 模型总结 …… 21
8.1 模型的优点 …… 21
8.2 模型的缺点 …… 21
8.3 模型的改进与推广 …… 21
参考文献 …… 22
A 附录 所用软件 …… 23
B 附录 代码 …… 23

# 一、 问题重述

## 1.1 问题背景

近年来，共享单车已经成为高校学生通勤的重要工具，显著提升了出行便利性。但是在其运营的过程中，仍然暴露出了许多不合理的设计，比如投放点位设计不合理和高峰期运力不足等多个方面。为评估使用现状并优化运营方案，该校学生对若干停车点的单车数量进行了统计调查。

## 1.2 问题重述

**问题一**：基于附件中不完全的停车点单车数量统计，估算出校内共享单车总数及各停车点在不同时间点的车辆数量分布。

**问题二**：建立各停车点在不同时间的需求模型及调度模型，以缓解高峰期供需矛盾。

**问题三**：基于问题二的调度模型，综合多指标构建运营效率评价模型，评估现有停车点布局的合理性。若不合理，提出停车点的调整方案并重新评估。

**问题四**：基于问题三的优化布局，动态规划高效回收路径，在最短总耗时内将故障车辆比例降至最低，保障学生用车需求。

# 二、 问题分析

对于**问题一**，首先我们要建立每个停车点的单车需求量与空间、时间、天气和事件等因素之间的定量关系。通过自然三次样条插值法，根据表中的有效数据构建插值函数。接着，将不同时间值代入插值函数，计算各停车点在不同时间的共享单车总量。最终，选取一定时间点单车总数的最大值，以此估算校园内的共享单车总量。

对于**问题二**，首先需要建立考虑时间因素与空间因素的用车需求模型，为此我们采用了问题一建立的需求函数模型，对每个停车点代入相应的参数，求得其在特定时间段内具体的需求量。随后，我们构建基于时空约束的多车辆联合调度模型。建立一个元素为停车点之间的最短路径距离的矩阵，其中最短路径使用 Dijkstra 算法计算，从而把原图抽象为一个赋权完全图。接下来，建立调度路径的评估模型，并最终采用遗传算法求解该问题。

对于**问题三**，为了建立共享单车运营效率的评价模型，我们分为两个阶段进行建模。第一阶段，采用带评价准则的聚类算法生成候选停车点；第二阶段采用 TOPSIS 方法确定最优聚类数，从而最终确定合理的停车点设置方式。其中，最关键的步骤是如何选取并量化 TOPSIS 法的多个评价指标，以及如何正确地引入评价指标和分配合理的权重，以便有效运用带评价指标的 K-means 聚类模型进行求解。

对于**问题四**，我们需要优化巡检路线，以在最短时间内收集最多的故障单车。考虑到车辆容量限制、停车点之间的最短路径距离等约束，我们选择建立静态容量约束车辆路径问题（CVRP）模型。然后采用多趟容量约束车辆路径问题的贪心启发式算法，从维修点出发，逐步构建巡检路径，直到所有故障单车被收集。

# 三、 模型准备

## 3.1 模型假设

为简化问题，本文做出以下假设：

- 假设 1：本文假设模型运行过程中忽略天气条件对用户出行行为及调度路径可行性、安全性的影响。

• 假设 2：在任意观测时刻，仅有少量电动车处于行驶状态，大多数电动车停放于各停车点。因此，采集到的停车点状态数据可近似视为该时刻电动车空间分布的合理反映。

• 假设 3：高峰时段内的车辆需求可由历史平均值或基于时间序列的预测模型进行估算，并假设实际需求量与预测值一致。模型中不考虑诸如车辆短缺对后续需求产生的二次反馈效应。

• 假设 4：假设研究周期内研究区域的道路结构保持不变，图模型中各节点（停车点）与边（道路）连接关系稳定可用，不因施工、交通管制等外部因素发生动态变化。

### 3.2 符号说明

| 符号 | 说明 | 单位 |
|---|---|---|
| $\hat{x}_k(t)$ | 单车需求总量 | 辆 |
| $f_k(t)$ | 需求函数 | 1 |
| $G(V,E)$ | 顶点集为 $V$，边集为 $E$ 的图 | - |
| $\Delta_{i,t}$ | 节点 $i$ 在 $t$ 时刻真实供需量 | 辆 |
| $F(S)$ | 遗传算法适应度函数 | - |
| $d_{i,j}$ | 两个停车点之间的距离 | m |
| $D_{i,j}$ | 距离矩阵 | m |
| $\tilde{D}_{i,j}$ | 遗传算法协同距离矩阵 | m |
| $\eta$ | 调度效率 | 1 |
| $SSE(K)$ | 供需匹配率 | 1 |

* 文中其余符号将在相应位置进行说明。

## 四、 问题一的模型建立与求解

### 4.1 问题一的模型建立

根据”附件 1-共享单车分布统计表.xlsx”与”附件 2 校园地图.pdf”，我们建立了如下模型。集合 $\mathcal{P}=\{P_1,P_2,\ldots,P_N\}$ 表示 $N=15$ 个停车点，其中第 $k$ 个停车点 $P_k$ 的坐标为 $(x_{P_k},y_{P_k})$。一系列的观测时间记为 $\{t_1,\ldots,t_m\}$，根据附件一获得的数据集：$(t_{i,k},x_{i,k})$，其中 $t_i$ 表示观测时间，$x_{i,k}$ 表示在 $t_i$ 时停车点 $P_k$ 处的单车数量。

在任一时间点 $t$，我们假设停车点 $P_k$ 的单车需求 $\hat{x}_k(t)$ 满足如下关系：

$$\hat{x}_k(t)=f_k(t)\cdot C_k \tag{1}$$

其中 $C_k$ 表示 $P_k$ 的理论最大停车位数，$f_k(t)$ 是需求函数，依赖于停车点的位置、时间、周周期、季节/天气、外部事件等因素。为便于描述，我们给出一个关于这些因素的典型需求函数定义如下：

需求函数 $f_k(t)$ 定义为空间、时间、天气和事件因子的乘积形式：

$$f_k(t)=\underbrace{\Phi_{\text{space}}}_{\text{位置}}\cdot\underbrace{\Psi_{\text{time}}(t)}_{\text{时间周期}}\cdot\underbrace{\Omega_{\text{weather}}(t)}_{\text{天气}}\cdot\underbrace{\Gamma_{\text{event}}(t)}_{\text{事件}} \tag{2}$$

其中，各因子的定义如下：

1. **空间位置因子** $\Phi_{\text{space}}$ 可以通过改进的引力模型进行衡量：

$$\Phi_{\text{space}}=A\cdot\left(\sum_{i=1}^{N}\frac{w_i\cdot P_i}{d_i^{\alpha}}\right)^{\beta}$$

其中：

- $P_i$: 设施 $i$ 的人流量（单位：人次/小时）
- $d_i$: 到设施 $i$ 的欧氏距离（单位：m）
- $w_i$: 权重系数（如教学楼 $w = 1.2$，食堂 $w = 0.8$）
- $\alpha, \beta$: 衰减参数（$\alpha \in [1.5, 2.5]$）
- $A$: 归一化常数

2. **时间周期因子** $\Psi_{\text{time}}(t)$ 融合了日内周期、周周期和节假日因素：

$$\Psi_{\text{time}}(t) = \exp\left(-\frac{(t_{\text{hour}} - \mu)^2}{2\sigma^2}\right) \cdot \left[1 + \gamma \sin\left(\frac{2\pi t_{\text{week}}}{7} + \phi\right)\right] \cdot (1 - \eta \cdot H(t))$$

其中：

- $t_{\text{hour}} \in [0, 24)$: 当前小时
- $\mu = 8, \sigma = 1.5$: 早高峰参数
- $H(t) \in \{0, 1\}$: 节假日指示函数

3. **天气因子** $\Omega_{\text{weather}}(t)$ 考虑了温度、降雨、风速的非线性耦合：

$$\Omega_{\text{weather}}(t) = \exp\left(\theta_1 T(t) - \theta_2 R(t)\right) \cdot (1 + \theta_3 \ln(1 + S(t)))$$

其中：

- $T(t)$: 标准化温度（$T \in [-1, 1]$）
- $R(t) \leq 50$: 降雨量（单位：mm/h）
- $\theta_1 > 0, \theta_2 > 0$: 温度和降雨的敏感系数

4. **事件因子** $\Gamma_{\text{event}}(t)$ 描述了脉冲事件的时间衰减效应：

$$\Gamma_{\text{event}}(t) = 1 + \sum_{j=1}^{M} \kappa_j \cdot E_j(t_0^{(j)}) \cdot \int_{t_0^{(j)}}^{t} e^{-\lambda_j (t-\tau)}\, d\tau$$

其中：

- $E_j(t_0^{(j)})$: 事件强度（如考试周 $E = 0.8$）
- $\lambda_j$: 衰减率（单位：$\lambda = 0.1\,\text{h}^{-1}$）
- 积分核用于描述事件的记忆效应

基于上述模型，我们认为，在一般情况下，停车点 $P_k$ 的单车需求 $w_k(t)$ 与停车点的观测值 $r_{i,k}$ 大致相同。由于 $f_k$ 可视作闭区间内的连续函数，故由 Weierstrass 第一逼近定理，可以对 $f_k$ 进行多项式逼近，这意味着 $r_{i,k}$ 可以通过关于时间参数 $t_i$ 的多项式来逼近。

在此建模假设下，我们可以得出全校的总单车数量 $R$ 为：

$$R = \max_k \sum_{k=1}^{N} r_{i,k} \tag{3}$$

### 4.2 问题一的求解：三次样条插值

在前文中，我们提到，根据对若干停车点单车数量的统计数据，我们可以利用多项式来估算不同时间点停车点的共享单车数量分布。为此，我们选择了自然三次样条插值方法来逼近实际函数，具体原因将在后文说明。

常见的插值方法包括多项式插值、分段插值和三次样条插值等。多项式插值如 Lagrange、Newton 容易在区间端点产生剧烈震荡（龙格现象），不适用于插值点较多的情况。分段插值方法虽稳定，但多数依赖于已知的一阶导数值，而本问题中无法获取，因此同样不适用。

相比之下，三次样条插值是一种较为稳妥的方法，尤其是在数据点较多且导数值未知的情况下。自然三次样条插值是一种特别适用于边界变化较为平缓的问题的插值方法，其特点是保证插值函数在区间内具有连续的函数值、一阶导数和二阶导数。具体到本问题，考虑到停车点的变化通常在周三的 0 点和周日的 24 点较为平缓，因此我们选用自然三次样条插值最为合适。

**基于自然三次样条插值的求解过程**

为了实现自然三次样条插值，我们采用了 MATLAB 中的“csape”函数。实现步骤如下：

- 首先读取停车点的数据。在 MATLAB 中，我们将 15 个停车点在五天内的数据（即 D2:D35、E2:E35，依此类推，直到 R2:R35）分别存储在向量“vector_D”、“vector_E”等中。此过程中，需要注意忽略缺失值，并将表格中所有显示为 200+ 的数值统一视为 200。
- 其次读取时间数据（C2:C35），并进行适当的处理，存储为“time_vector”。考虑到最终插值的时间轴是从周三的 0 点到周日的 24 点，因此需要将时间数据转换为一个标准化的时间向量，范围从 0 到 5，其中周三 0 点作为时间轴的原点，周日 24 点对应时间轴上的 5。
- 接下来，将每个停车点的数据与对应的时间向量匹配，使其与原表格中的共享单车数量一一对应。
- 然后对时间向量和停车点数据进行边界值的补充。具体地，我们为每个时间向量的起始和结束分别添加 0 和 5，并为每个停车点数据向量的起始和结束分别添加假定值，以模拟边界条件。这样做的目的是假设所有数据点的边界二阶导数为零，从而符合自然三次样条插值的要求。
- 最后，利用 MATLAB 中的“csape”函数对每一组匹配数据进行插值，得到 15 个结构体，分别对应不同停车点的插值结果。通过“fnval”函数，我们可以计算在给定时间点对应的共享单车数量，并对五天数据进行平均，最终填入表1中。

| | 7:00 | 9:00 | 12:00 | 14:00 | 18:00 | 21:00 | 23:00 |
|---|---|---|---|---|---|---|---|
| **东门** | 37 | 52 | 67 | 74 | 88 | 80 | 55 |
| **南门** | 59 | 55 | 52 | 59 | 95 | 81 | 65 |
| **北门** | 23 | 62 | 67 | 66 | 74 | 44 | 25 |
| **一食堂** | 36 | 35 | 47 | 16 | 0 | 35 | 69 |
| **二食堂** | 116 | 24 | 88 | 153 | 94 | 71 | 93 |
| **三食堂** | 44 | 36 | 39 | 34 | 44 | 65 | 83 |
| **梅苑 1 栋** | 87 | 42 | 48 | 23 | 0 | 29 | 92 |
| **菊苑 1 栋** | 135 | 115 | 89 | 78 | 85 | 99 | 105 |
| **教学 2 楼** | 166 | 190 | 56 | 0 | 41 | 72 | 73 |
| **教学 4 楼** | 18 | 61 | 94 | 95 | 68 | 36 | 6 |
| **计算机学院** | 0 | 20 | 10 | 37 | 51 | 24 | 6 |
| **工程中心** | 60 | 56 | 27 | 24 | 48 | 57 | 50 |
| **网球场** | 7 | 18 | 22 | 17 | 19 | 12 | 7 |
| **体育馆** | 12 | 8 | 5 | 10 | 31 | 34 | 36 |
| **校医院** | 12 | 13 | 16 | 16 | 11 | 8 | 14 |

**表 1　共享单车分布统计表**

由上表，计算得出每一时间点各个停车点停放单车数量的和的最大值为 775，于是我们对校园内总车辆数提出 775 **辆的下界估计**。根据此表，我们还能得到可视化图如图1。该图形象地展示了车辆分布随时间的变化。

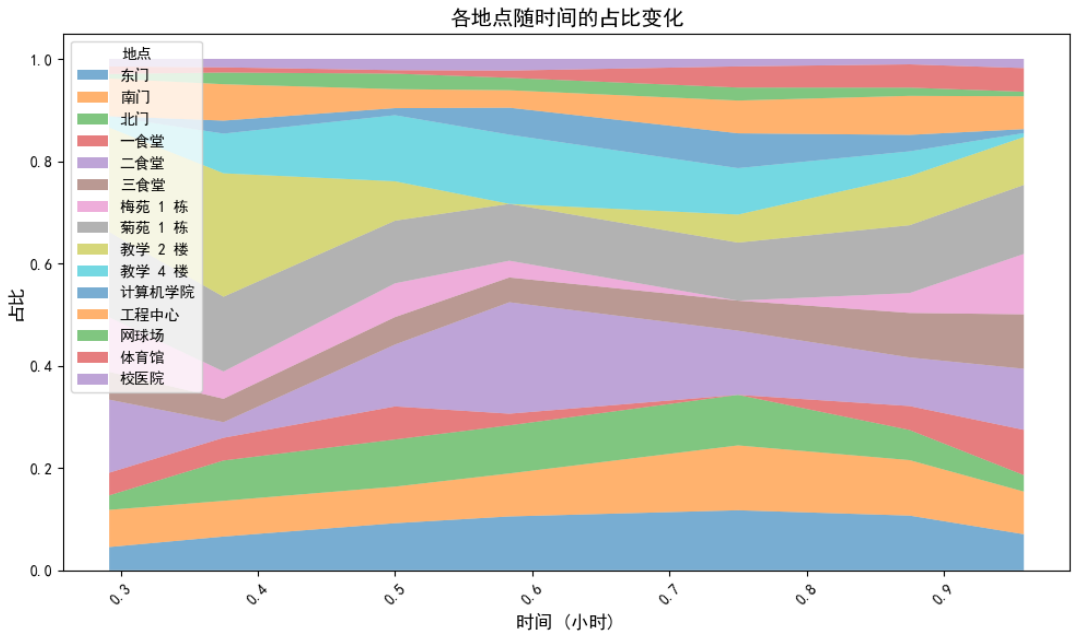



**图 1　问题一结果可视化**

# 五、 问题二的模型建立与求解

## 5.1 问题二的模型建立

### 5.1.1 关于时间空间的用车需求模型

$f_k(t)$ 可以通过公式 (2) 直接计算得到，然后通过合理选择参数，我们可以推算出该位置的用车需求量。以下是一个具体示例：

> 假设某停放点附近有教学楼（$P = 500$ 人/h，$d = 200$ m），早高峰时段，$\mu = 8$，温度为 $T = 20°\mathrm{C}$，且考试周事件影响因子 $E = 0.8$，则模型的预测值为：

$$f_k(t) \propto \left(\frac{1.2 \times 500}{200^{1.8}}\right)^{1.1} \cdot \mathrm{e}^{-\frac{(8-8)^2}{2\times 1.5^2}} \cdot \mathrm{e}^{0.05\times 20} \cdot \left(1 + 0.2\int_{t_0}^{t} \mathrm{e}^{-0.1(t-\tau)}\, d\tau\right) \approx 12.7\,(\text{人/小时})$$

根据公式 (2) 并结合上述参数的合理选取，我们可以计算出每个停车点在特定时间的需求量，具体结果见表 2。

| 地点 | 7:00 | 12:00 | 18:00 | 地点 | 7:00 | 12:00 | 18:00 |
|---|---|---|---|---|---|---|---|
| 东门 | 19 | 82 | 66 | 教学 2 楼 | 73 | 190 | 31 |
| 南门 | 53 | 55 | 45 | 教学 4 楼 | 6 | 61 | 63 |
| 北门 | 21 | 62 | 36 | 计算机学院 | 6 | 2 | 10 |
| 一食堂 | 69 | 35 | 51 | 工程中心 | 50 | 8 | 33 |
| 二食堂 | 93 | 24 | 130 | 网球场 | 7 | 18 | 14 |
| 三食堂 | 49 | 36 | 75 | 体育馆 | 36 | 8 | 32 |
| 梅苑 1 栋 | 92 | 42 | 58 | 校医院 | 14 | 19 | 5 |
| 菊苑 1 栋 | 105 | 115 | 106 | | | | |

**表 2　不同高峰期各个停车点的需求量**

### 5.1.2 关于时空的多车辆联合调度模型建立

**（一）构建站点距离矩阵**　为了方便表示，我们构建了一个站点距离矩阵 $D = \left[d_{i,j}\right]_{N\times N}$，该矩阵的元素 $d_{i,j}$ 表示站点 $i$ 和站点 $j$ 之间的距离，这个距离指的是两个停车点之间的最短路径，$N$ 为停车点的数量。

这里我们使用 Dijkstra 算法（算法1）来计算在这类图上的多源最短路径问题。

**Algorithm 1** Dijkstra 算法（基于站点距离矩阵 $D$）

**输入:** 站点距离矩阵 $D = [d_{i,j}]$, 源点 $s \in V$
**输出:** 最短路径数组 $d$, 前驱节点数组 $p$

```
初始化：dis[v] = ∞, ∀v ∈ V，d[s] = 0
初始化：pre[v] = null, ∀v ∈ V
初始化：优先队列 Q = V
while Q ≠ ∅ do
  从 Q 中取出 u，使得 dis[u] 最小
  for 每个邻接点 v of u do
    if v ∈ Q 且 dis[u] + d_{u,v} < dis[v] then
      dis[v] = dis[u] + d_{u,v}
      pre[v] = u
      将 v 更新到优先队列中
    end if
  end for
end while
```

计算后，我们便可以将整张图抽象为一个 $N$ 个点，$N^2$ 条边的添加了环的赋权完全图 $G(V, E)$，其中 $V$ 是 $N$ 个停车点构成的集合，$E$ 是 $N^2$ 条边构成的边集合。

**（二）调度路径评估模型** 设站点 $i$ 在时刻 $t$ 的真实供需量为 $\Delta_{i,t}$:

$$\Delta_{i,t} = x_{i,t} - \hat{x}_{i,t}$$

若 $\Delta_{i,t} > 0$，则表示该站点 $i$ 在 $t$ 时刻有盈余车辆，可以调走, 并将该点纳入车辆盈余集 $O$；若 $\Delta_{i,t} < 0$, 则表示车辆短缺，需要调入，并将该点纳入车辆短缺集 $S$。

设 $S_v$ 为第 $v$ 辆车的调度方案：

$$S_v = p_0, p_1, p_2 \cdots p_{n_v}$$

为记录单车的调度情况，$n_v$ 为调度车 $v$ 的路径节点数。，另设 $C_{i,j,v}$ 表示调度车 $v$ 运送从站点 $i$ 向站点 $j$ 运送的单车数，可以得到单车运输量的最大值：

$$\max \sum_{v=1}^{3} \sum_{j \in S} \sum_{i \in O} C_{i,j,v}$$

在最大化运输量的同时，需考虑调度路径的总长度以降低时间与能耗成本。设调度车 $v$ 的总行驶距离为:

$$\sum_{k=0}^{n_v - 1} d_{p_k, p_{k+1}}$$

其中 $d_{p_k, p_{k+1}}$ 为站点 $p_k$ 到 $p_{k+1}$ 的距离（来自矩阵 $D$）。

将最大化运输量与最小化路径长度结合，定义综合目标函数：

$$\text{maximize} \left( \alpha \sum_{v=1}^{3} \sum_{i \in O} \sum_{j \in S} C_{i,j,v} - \beta \sum_{v=1}^{3} \sum_{k=0}^{n_v - 1} d_{p_k, p_{k+1}} \right) \tag{4}$$

其中：$\alpha, \beta \geq 0$ 为权重系数，反映运输效率与路径成本的优先级。

**约束条件**:

1. **单次调度操作限制**（避免超调）：

$$C_{i,j,v} \leq \Delta_{i,t}, \qquad \forall i \in O, j \in S, v \in \{1,2,3\} \tag{5}$$

$$C_{i,j,v} \leq |\Delta_{j,t}|, \qquad \forall i \in O, j \in S, v \in \{1,2,3\} \tag{6}$$

这一限制指出：

- 式(5)：车辆 $v$ 从站点 $i$ 调出的单车数不超过其盈余量 $\Delta_{i,t}$。
- 式(6)：车辆 $v$ 调入站点 $j$ 的单车数不超过其短缺量 $|\Delta_{j,t}|$。

2. **车辆总容量限制**（每辆车不超载）：

$$\sum_{i \in O} \sum_{j \in S} C_{i,j,v} \leq 20, \quad \forall v \in \{1,2,3\} \tag{7}$$

3. **时间窗口约束**

$$\sum_{k=0}^{n_v-1} \frac{D_{p_k,p_{k+1}}}{v} \leq T_{\max}, \quad \forall v \in \{1,2,3\} \tag{8}$$

其中：

- $D_{p_k,p_{k+1}}$ 为路径中相邻站点间的距离（单位：m）。
- $T_{\max}$ 为调度车必须在高峰前完成运输的最大允许行驶时间（单位：min）。
- $v$ 为调度车的运行速度（单位：m/min）。

### 5.1.3 基于遗传算法的模型求解

1. **算法设计框架**

为实现多车辆协同调度优化，我们采用遗传算法解决多辆调度车的路径规划问题，核心思想是通过模拟生物进化过程，逐步优化车辆的调度方案。其核心模块如下：

- **访问序列编码**：调度方案编码为三元组 $S = (S_1, S_2, S_3)$，其中每辆车路径 $S_v$ 定义为操作序列：

$$S_v = \langle (p_1, \Delta c_1), (p_2, \Delta c_2), \ldots, (p_n, \Delta c_n) \rangle$$

其中 $p_k$ 为第 $k$ 步访问的站点编号，$\Delta c_k \in [-20, 20]$ 为调运量（负值表示调出）。

- **协同约束机制**：全局状态实时更新约束车辆操作：

$$\sum_{v=1}^{3} C_{i,j,v} \leq \Delta_{i,t} \quad (\forall i \in O), \qquad \sum_{v=1}^{3} C_{i,j,v} \leq |\Delta_{j,t}| \quad (\forall j \in S)$$

- **适应度函数**：尽量减少各站点的车辆短缺或过剩，同时缩短三辆车的总行驶距离。综合评估后：

$$F(S) = \underbrace{\sum_{i=1}^{N} (\text{短缺惩罚}(i) + \text{盈余惩罚}(i))}_{\text{供需平衡指标}} + \underbrace{\gamma \cdot \sum_{v=1}^{3} D(S_v)}_{\text{路径成本}} \tag{9}$$

其中：

- **短缺惩罚**：车辆短缺站点（$\Delta_{i,t} < 0$），采用惩罚函数：短缺惩罚$(i) = \begin{cases} (|\Delta_{i,t}|)^2, & \text{若}\Delta_{i,t} < 0 \\ 0, & \text{其他} \end{cases}$
- **盈余惩罚**：车辆过剩站点（$\Delta_{i,t} > 0$），采用惩罚函数：盈余惩罚$(i) = \begin{cases} \Delta_{i,t}, & \text{若}\Delta_{i,t} > 0 \\ 0, & \text{其他} \end{cases}$

- **路径成本**:三辆调度车的总行驶距离,由距离矩阵 $D(S_v)$ 计算,$\gamma$ 为权重系数。当 $\gamma = 1.0$ 时，供需平衡与路径成本权重相等。

2. **多车协同调度机制**
   三辆车的联合调度需满足以下规则:
   - **动态资源竞争**：若站点 $i$ 初始盈余为 $\Delta_{i,t}$，车辆 $v_1$ 调出 $c_1$ 辆后，车辆 $v_2$ 最大可调出 $\Delta_{i,t} - c_1$ 辆。
   - **路径耦合优化**：路径规划时避免冗余访问。定义协同距离矩阵 $\tilde{D}_{i,j}$ 为:

$$\tilde{D}_{i,j} = D_{i,j} \cdot (1 + \lambda \cdot \text{其他车辆访问频次}(j))$$

     其中 $\lambda$ 为路径重叠惩罚系数。
   - **时间窗口同步**：车辆总行驶时间 $T_v = \sum_k \frac{D_{p_k,p_{k+1}}}{v}$ 需满足:

$$\max(T_1, T_2, T_3) \leq T_{\max}$$

3. **遗传算法实现步骤**如算法 2所示。

**Algorithm 2** 多车辆协同调度遗传算法

**输入**: 站点供需数据 $\Delta_{i,t}$, 距离矩阵 $D_{i,j}$, 参数 $K, G, \gamma$
**输出**: 最优调度方案 $S^*$

1. 初始化: 生成 $K$ 个随机个体，每个体包含三辆车独立路径
2. 迭代优化:
**for** $g = 1$ to $G$ **do**
  a. **适应度计算**: 按式 (9) 评估所有个体
  b. **锦标赛选择**: 选取适应度前 30% 的个体作为父代
  c. **路径交叉**: 以概率 $p_c$ 交换两父代的车辆路径
  d. **变异操作**: 以概率 $p_m$ 随机修改某车辆的站点或调运量
**end for**
3. **return** 适应度最优的个体 $S^*$

关键参数配置如表 3 所示。

| 参数 | 符号 | 取值 |
|---|---|---|
| 种群规模 | $K$ | 20 |
| 进化代数 | $G$ | 8000 |
| 交叉概率 | $p_c$ | 0.7 |
| 变异概率 | $p_m$ | 0.1 |
| 距离惩罚系数 | $\gamma$ | 0.2 |

**表 3　遗传算法参数配置**

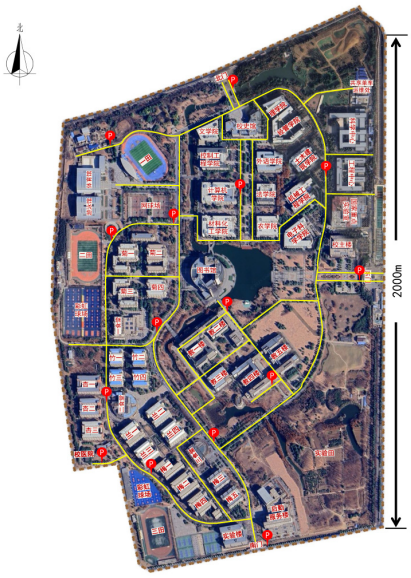
## 5.2 问题二的求解

### 5.2.1 地图信息的离散化建图过程

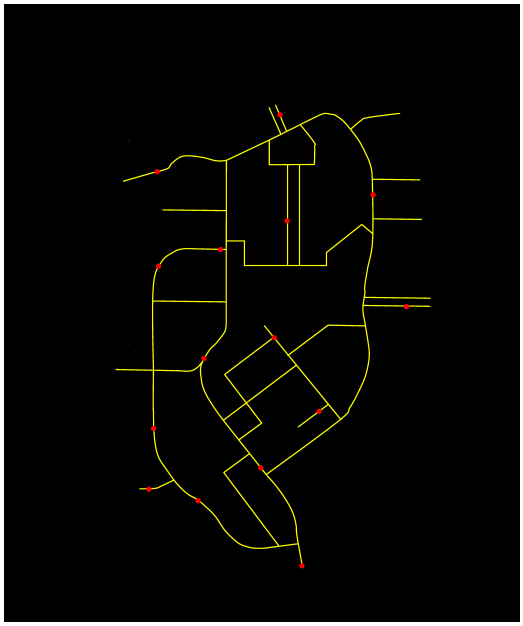
本小节旨在将提供的原始地图图像转化为可用于路径规划与调度优化的离散图结构。该过程包括道路与停车点的提取、离散节点的构建以及连接关系的定义。为此，我们基于计算机视觉技术对图像进行处理，采用了开源视觉库 **OpenCV** 中的相关函数进行颜色空间转换和区域识别，自动化提取图像中的关键交通元素。具体处理步骤如下:

- 首先将原始 RGB 图像转换为 HSV 色彩空间，以增强颜色分离能力，并设定颜色阈值以提取图像中所有黄色区域，作为候选道路像素；
- 其次提取红色区域，在此基础上对每个连通区域进行面积与形状分析：保留面积处于合理范围且轮廓近似椭圆的区域，作为停车点；

提取结果如图 2 所示：图 2a 为原始校园地图，图 2b 为识别出的道路与停车点结果。其中黄色线条表示可通行路径，红色点表示有效识别的停车点位置。




(a) 校园地图



(b) 提取的道路与停车点

**图 2　问题二中地图的处理与建图结果**

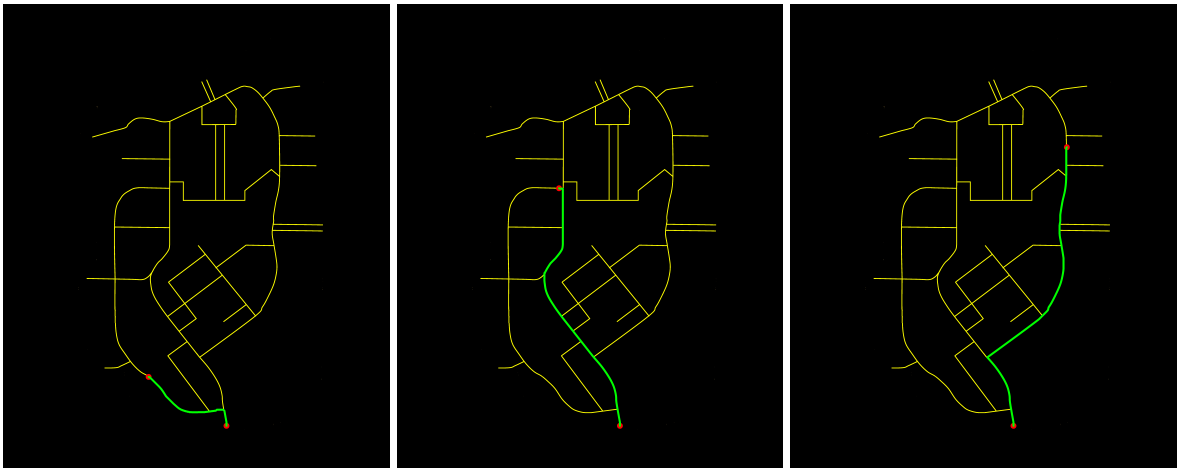
在此基础上，为将停车点准确纳入路径图结构，我们对每个停车点进行最近投影操作：将其在图像坐标上投射至最近的道路像素，从而使每一个停车点都能对应路径网络中的有效节点，于是可以建立图结构 $G(V, E)$，为后续路径计算奠定结构基础。

### 5.2.2 多源最短路径计算与距离矩阵构建

完成建图后，我们在生成的图结构 $G(V, E)$ 上执行多源最短路径计算，以构造停车点之间的距离矩阵 $D = [d_{i,j}]_{N\times N}$。考虑到图的相对稀疏性，我们采用 **Dijkstra 算法**，在每个停车点为源点的基础上，分别计算其到所有其他点的最短路径。为提升计算效率，我们在实现中引入了以下优化策略：

- **单源复用：** 由于从一个源点出发的最短路径可以一次性得到所有目标点距离，故每个节点仅需调用一次 Dijkstra 算法，从而将总调用次数从 $\mathcal{O}(N^2)$ 降低至 $\mathcal{O}(N)$；
- **对称性利用：** 图 $G$ 为无向图，节点对 $(i, j)$ 与 $(j, i)$ 的最短路径相同，故实际只需计算矩阵的上三角部分 $(i = 1 : n, j = i + 1 : n)$；
- **并行加速：** 利用 Python 中 `concurrent.futures` 模块中的 `ProcessPoolExecutor()` 实现并行处理任务，使算法能够充分利用多核 CPU，显著缩短运行时间。

图 3 展示了从 0 号停车点出发至其他部分节点最短路径的可视化结果，路径以绿色高亮标注，可清晰观察其分布趋势。



(a) 从点 0 到点 1　　(b) 从点 0 到点 10　　(c) 从点 0 到点 12

**图 3　从 0 号停车点出发至其他部分节点的最短路径示意图**

根据地图的实际比例尺，原图每 3770 个像素约等于实际距离 2000 米，折算后得出换算比例为：每 1.835 个像素 ≈ 1 米。我们将像素距离通过该比例换算为实际距离，最终得到如图 4 所示的最短路径距离矩阵（单位为米，四舍五入为整数），该矩阵为后续车辆调度与路径规划提供了基础数据支撑。

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 600 | 852 | 465 | 985 | 949 | 1021 | 1249 | 1517 | 1623 | 1522 | 2072 | 1789 | 2181 | 2239 |
| 1 | | 0 | 241 | 873 | 377 | 1354 | 845 | 1611 | 1928 | 1040 | 1328 | 1898 | 2188 | 2001 | 2064 |
| 2 | | | 0 | 1124 | 360 | 1606 | 822 | 1591 | 2119 | 1018 | 1314 | 1877 | 2273 | 1990 | 2035 |
| 3 | | | | 0 | 952 | 548 | 551 | 778 | 1119 | 1149 | 1054 | 1598 | 1392 | 1711 | 1763 |
| 4 | | | | | 0 | 1334 | 463 | 1225 | 1756 | 663 | 949 | 1517 | 1909 | 1621 | 1678 |
| 5 | | | | | | 0 | 929 | 410 | 820 | 1541 | 1435 | 1599 | 1085 | 2100 | 1807 |
| 6 | | | | | | | 0 | 821 | 1356 | 663 | 501 | 1053 | 1445 | 1155 | 1209 |
| 7 | | | | | | | | 0 | 714 | 1422 | 1326 | 1499 | 983 | 1984 | 1711 |
| 8 | | | | | | | | | 0 | 1632 | 1343 | 1125 | 611 | 1876 | 1342 |
| 9 | | | | | | | | | | 0 | 285 | 874 | 1271 | 986 | 1037 |
| 10 | | | | | | | | | | | 0 | 590 | 981 | 699 | 754 |
| 11 | | | | | | | | | | | | 0 | 769 | 912 | 553 |
| 12 | | | | | | | | | | | | | 0 | 1254 | 721 |
| 13 | | | | | | | | | | | | | | 0 | 683 |
| 14 | | | | | | | | | | | | | | | 0 |

**图 4　Dijkstra 得到的距离矩阵（单位 m）（暂时表示为上三角矩阵方便观察）**

### 5.2.3 运用遗传模型的问题二结果

在实际调度问题中，为了兼顾效率与校园运行的正常秩序，我们结合供需关系分布、车辆初始状态以及学校作息时间，确定了三个主要的车辆调度时间段：6:00 ～ 7:00、12:30 ～ 13:30 和 16:00 ～ 17:00。此选择旨在于尽量避开早上、午后与傍晚的通勤高峰时段（分别为 7:30、14:00 和 18:00），确保调度操作能在高峰开始前高效完成，最大程度降低对通行秩序的干扰，并提升系统对高峰期用车需求的响应能力。

我们将前述计算得到的距离矩阵 $D$ 与供需信息参数 $\Delta_{i,t}$ 一并输入遗传算法模型中作为调度优化的基础数据。模型目标函数旨在最小化总调度距离，同时满足车辆容量约束与各站点的供需平衡要求。经过多轮迭代进化，遗传模型在每个指定调度时段内输出了对应的最优路径与车辆操作方案，结果如下：

1. **第一次调度**（6:00 ～ 7:00）：

遗传算法输出了三辆调度车的具体行驶路径与装载计划，详见如下的输出，其对应的可视化调度路径展示如图 5 所示，调度后相关指标（供需平衡等）的变化情况如图 6 所示。

```
Vehicle 0:
  Station 4: -18 cars (current load: 18)
  Station 3: +18 cars (current load: 0)
  Station 10: +0 cars (current load: 0)
  Station 0: +0 cars (current load: 0)
  Station 2: -2 cars (current load: 2)
  Total distance: 4377

Vehicle 1:
  Station 7: -12 cars (current load: 12)
  Station 1: -6 cars (current load: 18)
  Station 13: +18 cars (current load: 0)
  Station 7: -18 cars (current load: 18)
  Station 5: +18 cars (current load: 0)
  Total distance: 6005

Vehicle 2:
  Station 4: -5 cars (current load: 5)
  Station 2: +0 cars (current load: 5)
  Station 0: -9 cars (current load: 14)
  Station 8: -4 cars (current load: 18)
  Station 3: +18 cars (current load: 0)
  Total distance: 3847
```

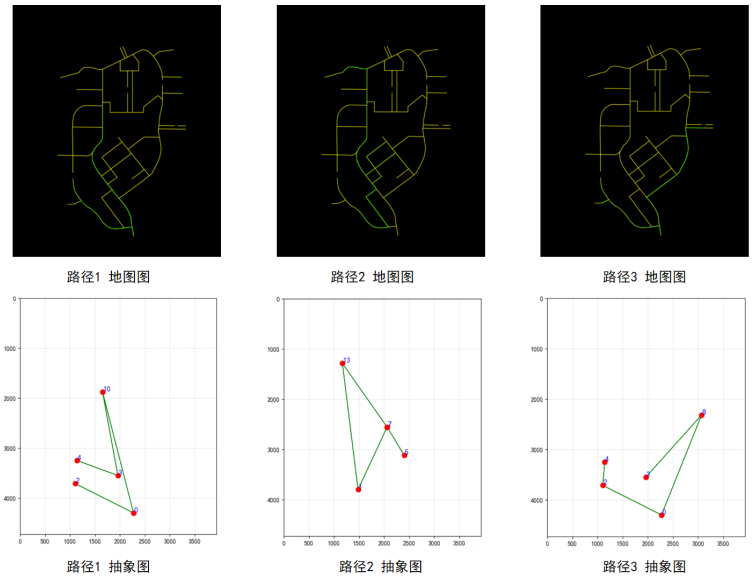



**图 5　6:00 ∼ 7:00 调度车辆的路径可视化**

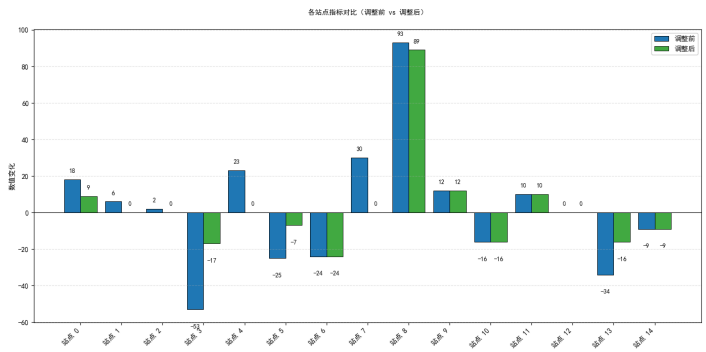



**图 6　6:00 ∼ 7:00 调度完成后的指标变化**

2. **第二次调度**（12:30 ∼ 13:30）：
同样地，遗传算法输出了符合供需匹配与路径优化的调度方案。出于篇幅考虑，路径表内容此处省略，仅保留两张核心图示：图 7 为调度路径可视化结果，图 8 展示调度执行后的

指标改变量。

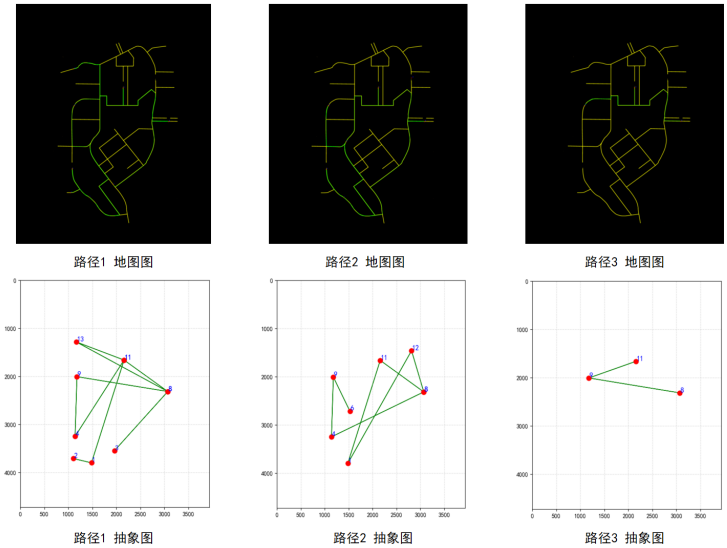



**图 7** 12:30 ∼ 13:30 **调度车辆的路径可视化**

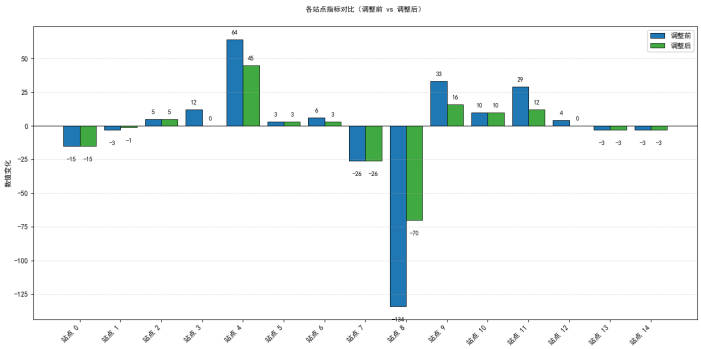



**图 8** 12:30 ∼ 13:30 **调度完成后的指标变化**

3. **第三次调度**（16:00 ∼ 17:00）:
最后一个调度周期同样遵循优化模型的输出，车辆路径与供需调整效果通过图 9 与图 10 展示。模型在该时段亦实现了良好的运输效率与供需平衡。

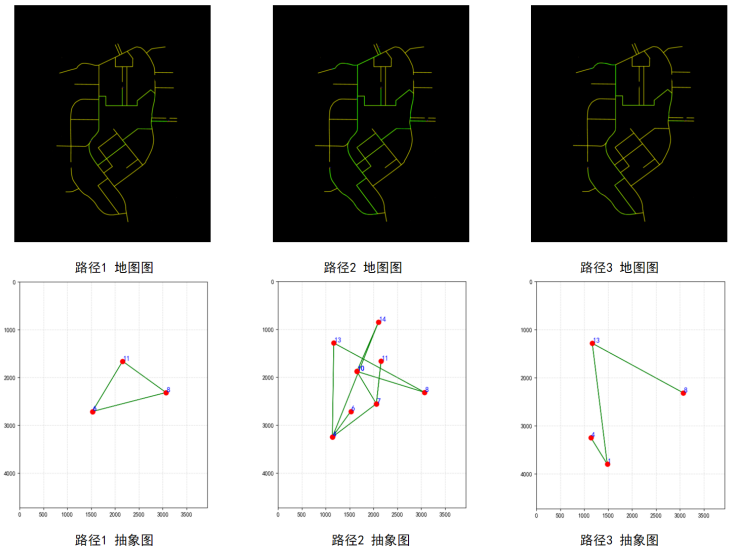



**图 9** 16:00 ∼ 17:00 **调度车辆的路径可视化**

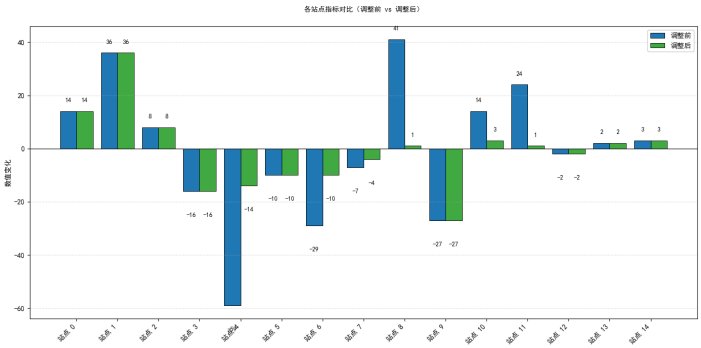



**图 10**　16:00 ∼ 17:00 **调度完成后的指标变化**

### 5.3 问题二模型的评价

在前述研究中，我们基于实际供需数据、停车点间的最短路径距离矩阵以及学校的作息规律，构建了一个面向时段调度优化的遗传算法模型。该模型成功在三个关键时段内生成了符合约束条件的高质量调度路径。为进一步量化方案的调度效果，我们在本节中尝试引入一个评价指标，对调度路径的执行效率进行评估与分析。

我们定义路径的调度效率为：

$$\eta = 1 - \frac{\sum \left(\Delta^{-}_{\text{after}}\right)^2 + \sum \left|\Delta^{+}_{\text{after}}\right|}{\sum \left(\Delta^{-}_{\text{before}}\right)^2 + \sum \left|\Delta^{+}_{\text{before}}\right|} \times 100\%,$$

其中 $\Delta^{+}$ 与 $\Delta^{-}$ 分别表示调度前后各点位的剩余供需量中为正值与负值的部分。对于负值部分（即缺车点位），我们采用平方和的形式以增强其在效率评估中的权重，体现模型对“缺口”弥补的优先关注；而正值部分则使用绝对值求和反映资源过剩所带来的效率损失。

应用上述评价公式，我们对三个调度时间段下的路径执行效果进行计算。结果显示，在满足车辆容量限制与时间窗口约束的前提下，模型分别在 6:00 ∼ 7:00、12:30 ∼ 13:30 与 16:00 ∼ 17:00 时段中达成了 **71.44%、68.96% 与 73.78%** 的调度效率（$\eta$）。这些结果表明，在当前建模条件与数据输入下，遗传算法具有一定的优化能力，能够在复杂调度环境中提供相对合理的解决方案。尽管仍存在进一步提升空间，但该模型为后续优化方向提供了良好的实验基础。

## 六、 问题三的模型建立与求解

### 6.1 问题三的模型建立

本模型我们采用 **K-Means 聚类分析**和 **TOPSIS 优劣解距离法**实现共享单车停车点的科学规划，主要基于以下理论依据和方法论考量：

- **第一阶段：带评价准则的聚类算法生成不同停车点数的候选方案**
  1. 聚类分析能够有效识别骑行需求的空间分布特征，符合共享单车使用具有空间聚集性的客观规律。
  2. 引入多目标评价准则，首先确保聚类结果在功能区覆盖率和需求满足度之间取得平衡。
  3. 该方法能够自适应地发现数据内在结构，避免人为设定停车点数量带来的主观偏差。
- **第二阶段：TOPSIS 方法确定最优聚类数**
  1. 多准则决策分析方法能够系统综合供需匹配率、运营成本因素和用户便利性等多维评价标准。
  2. 通过计算各方案与理想解的相对接近度，可量化比较不同聚类数方案的综合表现。

3. 该方法具有良好的鲁棒性，能够有效处理评价指标间的冲突问题。

- **两阶段串联优化架构优势**
  1. 解耦了空间划分和方案优选两个决策过程，降低了问题求解复杂度。
  2. 模块化设计增强了模型的可解释性和可扩展性，便于根据其他需求调整评价指标体系。

### 6.1.1 TOPSIS 评价指标的选取

对于本问题，我们选取了多个评价指标来评价一种停车点位设置的共享单车运营效率的合理性，具体评价指标如下：

1. **停车点的数量：**在车辆调度中，停车点的数量直接影响调度的复杂度。一般而言，停车点数量较少时，调度路线较为简单，有利于提升调度效率。
   **量化方法：**停车点数量可以通过统计方案中停车点的总数来衡量。
   设某个方案一共有 $n$ 个停车点，分别为 $p_1, p_2, \cdots, p_n$，则量化公式为：
   $$S_1 = n$$
2. **各个停车点之间的平均距离：**在车辆调度中，各停车点之间的距离越短，调度效率通常越高。因此，停车点之间的平均距离是一个重要的优化指标。
   **量化方法：**计算所有停车点之间两两最短路径的平均值。
   设某个方案一共有 $n$ 个停车点，分别为 $p_1, p_2, \cdots, p_n$，则记 $d_{ij}$ 为 $p_i$ 到 $p_j$ 之间的最短路径距离。量化公式为：
   $$S_2 = \frac{1}{\binom{n}{2}} \sum_{1 \le i < j \le n} d_{ij}$$
3. **停车点到重要位置之间的距离：**在改变停车点位置的时候，需要考虑停车点到重要位置（如宿舍、食堂、教学楼）之间的距离，以免为了方便车辆调度而增加学生停车后的步行距离。
   **量化方法：**选取了菊苑宿舍中心点、一食堂竹苑中心点等 8 个重要位置以及北门、东门和南门这 3 个固定位置。在后续的方案考虑中，三个固定位置处必须有且仅有一个停车点。因此对于任意停车点方案，我们首先比较得出非固定位置的停车点距离哪一个重要（或固定）位置最短路径距离最小，就认为它属于哪个重要（或固定）位置对应的停车点。然后依次计算上述最短路径距离。
   设某方案共有 $n+3$ 个停车点，分别为 $p_1, p_2, \cdots, p_n, sp_1, sp_2, sp_3$，将八个重要位置从上到下分别命名为 $L_1, L_2, \cdots, L_8$。设 $p_i$ 到 $L_j$ 的路径距离为 $r_{ij}$，则停车点到重要位置之间的距离可量化为：
   $$S_3 = \frac{1}{n} \sum_{i=1}^{n} \min_{1 \le j \le 8} r_{ij}$$
4. **供需匹配率：**结合问题一中单车分布情况，衡量各停车点实际停放车辆数与时空需求模型预测值的匹配程度，避免车辆过剩或短缺。
   **量化方法：**对于单个停车点 $k$，计算其供需偏差的归一化值：
   $$Q_k = 1 - \frac{|A_k - w_k(t)|}{\max(A_k, w_k(t))}$$
   其中，$A_k$ 为新停车点 $k$ 的实际单车数，$w_k(t)$ 为时空模型预测的新停车点 $k$ 的需求量，$Q_k \in [0, 1]$ 值越大表示匹配度越高。综合所有停车点的匹配度，供需匹配率平均值为：
   $$S_4 = \frac{1}{n} \sum_{k=1}^{n} Q_k$$

5. **调度成本：**调度成本综合考虑了调度过程中的两方面因素：调度距离和调度单车数，最小化运输距离和数量，以降低调度成本。
   **量化方法：**基于问题二关于时空的多车辆联合调度模型，求出可被调动的单车的调度距离之和。得出调度总距离 $S_5$ 可以通过以下公式计算：

$$S_5 = \sum_{v=1}^{3}\sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij} \cdot m_{ij}$$

   其中，停车点 $d_{ij}$ 是 $i$ 和停车点 $j$ 之间的调度距离为，$m_{ij}$ 表示是从停车点 $i$ 向停车点 $j$ 调度的车数。如果不需要调度，$m_{ij} = 0$。

该评价方法基于 TOPSIS 多准则决策框架，首先构建原始数据矩阵 $X = [x_{ij}]_{m\times n}$，针对极小型指标通过 $x_{ij}^* = M - x_{ij}$ 进行正向化处理得到 $X^*$，随后采用向量归一化 $z_{ij} = x_{ij}^*/\sqrt{\sum_{k=1}^{m}(x_{kj}^*)^2}$ 获得标准化矩阵 $Z$；进而确定正负理想解 $Z_j^+ = \max z_{ij}$ 和 $Z_j^- = \min z_{ij}$，计算各方案到理想解的欧氏距离 $D_i^+ = \sqrt{\sum_{j=1}^{n}(z_{ij} - Z_j^+)^2}$ 和 $D_i^- = \sqrt{\sum_{j=1}^{n}(z_{ij} - Z_j^-)^2}$，最终通过相对接近度 $C_i = D_i^-/(D_i^+ + D_i^-)$ 实现方案排序，其中 $C_i \in [0,1]$ 值越大表明方案越接近理想状态。

### 6.1.2 带评价指标的 K-means 聚类模型

在本阶段，我们引入了供需匹配率和停车点到重要位置之间的距离这两个关键指标。与传统的 K-means 算法仅最优化空间距离不同，本模型采用了 TOPSIS 的思想，通过结合多个评价指标进行聚类分析，从而在不同站点数要求下，返回更符合实际需求的站点分布方案。

- **供需匹配率（权重 0.8）**：该指标用于衡量停车点与用车需求之间的匹配程度。由于供需匹配率仅与特定站点分布方案下的单车实际分布和真实需求情况相关，因此可以通过路径最短距离的平方和来计算：

$$\mathrm{SSE}(K) = \sum_{k=1}^{K}\underbrace{\sum_{p_i\in C_k} d(p_i, c_k)^2}_{\text{簇内路径距离平方和}}$$

  对原有数据进行处理得到 $d_{\mathrm{SSE}}(\cdot,\cdot)$。
- **停车点到重要位置之间的距离（权重 0.2）**：数据处理后得到 $S_3'$，用于评估停车点与关键位置之间的空间距离。

**Algorithm 3** 带评价指标的 K-Means 聚类算法（基于供需匹配率和可达性优化）

**输出:** 聚类中心点集合 $C = \{c_1, c_2, ..., c_K\}$, 聚类结果

初始化：随机选择 $K$ 个初始中心点 $C^{(0)} = \{c_1^{(0)}, c_2^{(0)}, ..., c_K^{(0)}\}$
计算每个数据点的初始聚类：$p_i \in C_k^{(0)}$
**for** $t = 1$ **to** 100 **do**
  **for** 每个数据点 $p_i$ **do**
    计算 $p_i$ 到每个聚类中心点的距离:$d(p_i, c_k) = 0.8 \cdot d_{\text{SSE}}(p_i, c_k) + 0.2 \cdot d_{S_3'}(p_i, c_k)$
    将 $p_i$ 分配到距离最小的聚类中心点：$p_i \in C_k^{(t)}$
  **end for**
  **for** 每个聚类中心点 $c_k$ **do**
    重新计算聚类中心：$c_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{p_i \in C_k^{(t)}} p_i$
  **end for**
  计算目标函数 $J^{(t)} = 0.8 \cdot \text{SSE}(K) + 0.2 \cdot S_3'$
  **if** $\frac{|J^{(t)} - J^{(t-1)}|}{J^{(t-1)}} < 0.01$ **then**
    **BREAK**
  **end if**
**end for**

### 6.2 问题三的求解：使用 TOPSIS 与 K-Means 综合确定最优站点分布方案

在确定了停车点的初步聚类模型后，为进一步优化聚类数量 $K$, 本阶段引入所有共 5 个指标，使用 TOPSIS 决策模型，对不同 $K$ 值下的聚类效果进行定量分析与比较，从而选出最优聚类数 $K$，以实现停车点布局的最优性与可实施性。指标的权重如表4所示。

| 序号 | 评价指标 | 符号 | 权重 |
|---|---|---|---|
| 1 | 停车点数量 | $S_1$ | 0.06 |
| 2 | 各停车点之间的平均距离 | $S_2$ | 0.04 |
| 3 | 到重要位置的距离 | $S_3$ | 0.13 |
| 4 | 供需匹配率 | $S_4$ | 0.42 |
| 5 | 调度成本 | $S_5$ | 0.35 |

**表 4　TOPSIS 评价指标及权重分配**

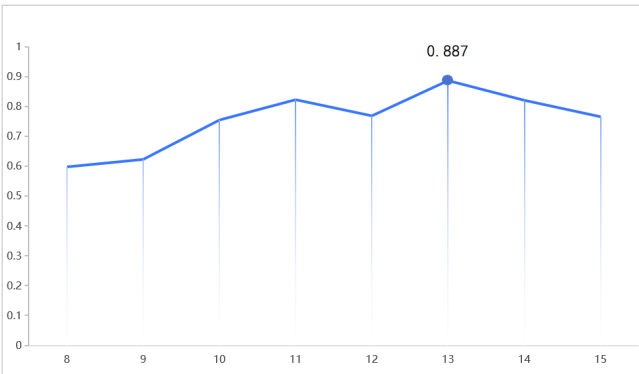


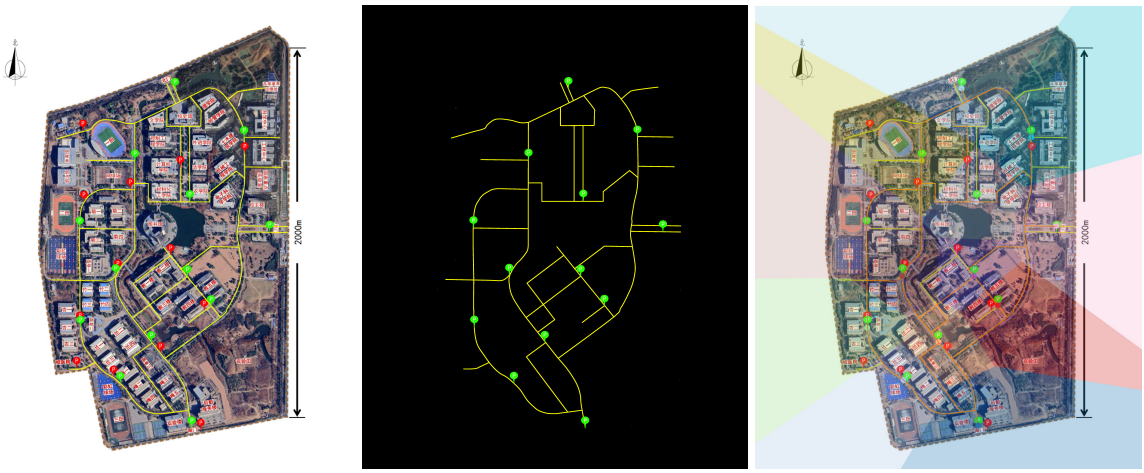

**图 11　TOPSIS 评价得出的不同聚类数得分**

根据图11可以看出，$K = 13$ 时得分最高，达到了 0.887，所以选定站点数为 13 时的分布方案为最优方案。

随后，我们将通过聚类算法得到的 13 个停车点的分布结果，映射到与每个停车点距离最

近的道路上。这样处理的目的是确保所有停车点最终都位于实际的道路网络上，以提高停车点的合理性和可操作性。

经过优化排序计算，本研究得出了站点空间分布的最优配置方案，如图12所示。具体而言：图12a直观呈现了优化后校园区域内停车点位的空间分布情况；图12b系统展示了基于路径网络分析提取的优化路径及相应停车点位；图12c则通过 Voronoi 空间分割模型，清晰表征了各空间点到最近停车点的服务范围边界，为评估停车设施的空间可达性提供了量化依据。




**(a) 更新后的学校停车点位示意图** **(b) 提取路径及停车点位示意图** **(c) Voronoi 图**

**图 12　更新后的停车点及路径图示**

在调整停车点布局后，供需匹配度从 72.4% **显著提升至 91.3%**。在问题二中，我们基于实际需求，构建了一个时空联合多车辆调度模型，该模型更加侧重于提高车辆调度数量，因此对调度距离的权重设置相对较低。经过停车点位的优化后，我们重新运行问题二中建立的模型，其调度总里程仅下降了 8.6%，但在早、中、晚三次高峰期的调度效率分别提升至 **84.12%**、**81.67% 和 92.96%**，分别提升了 17.75%、18.43%、26.00%。实现了显著优化。由此可见，停车点布局的调整显著提升了共享单车的运营效率，特别是在高峰时段的响应能力，充分验证了布局调整的合理性和效果。

# 七、 问题四的模型建立与求解

## 7.1 问题四的模型建立

本研究针对校园共享单车系统的维护优化问题，提出了一种多趟容量约束车辆路径问题（CVRP）模型。问题三优化停车点位之后，本系统包含 14 个节点：1 个维修点（节点 0）以及 13 个停车站点（节点 1 至 13）。每个停车站点存在一定数量的故障单车，需求量通过站点总单车数乘以 6% 的故障率并四舍五入得到。目标是利用一辆容量为 20 辆单车的维修车，最小化收集所有故障单车的总时间。每次巡检（单趟）需从维修点出发，收集不超过 20 辆故障单车后返回维修点，继续下一趟巡检，直至所有故障单车被收集。总时间包括行驶时间和搬运时间。

### 7.1.1 模型构建：考虑加权完成时间的静态 CVRP 模型

设路线条数（即往返次数）足够，对每一条路线 $k$ 定义二元决策变量

$$x_{ij}^{k} = \begin{cases} 1, & \text{若第 } k \text{ 条路线从节点 } i \text{ 直接驶向节点 } j, \\ 0, & \text{Otherwise.} \end{cases}$$

并令每条路线上的累计负荷不超过容量上限 $C = 20$：

$$\sum_{i=1}^{13} w_i\, y_i^k \;\le\; 20, \quad y_i^k = \begin{cases} 1, & \text{若第 } k \text{ 条路线访问了节点 } i, \\ 0, & \text{Otherwise.} \end{cases}$$

其中 $w_i$ 是该点的故障车辆数。

为捕捉“故障车等待时长”对目标的影响，引入节点 $i$ 在路线 $k$ 中的服务开始时间 $C_i^k$。若该路线按访问顺序 $(i_1, i_2, \ldots, i_m)$，则

$$C_{i_1}^k = T_{0,i_1}^{\text{drive}}, \tag{10}$$

$$C_{i_j}^k = C_{i_{j-1}}^k + T_{i_{j-1}}^{\text{service}} + T_{i_{j-1},i_j}^{\text{drive}}, \quad j = 2, \ldots, m. \tag{11}$$

其中

$$T_{a,b}^{\text{drive}} = \frac{d_{ab}}{v} \quad (\text{分钟}), \qquad T_i^{\text{service}} = w_i \quad (\text{分钟}).$$

其中 $v$ 为速度（25km/h = 146.67m/min）。

目标函数设为所有故障车加权完成时间之和最小值：

$$\text{minimize} \quad \sum_k \sum_{i=1}^{13} w_i\, C_i^k.$$

### 7.1.2 模型构建：多趟 CVRP 问题的贪心启发式算法

为了高效求解多趟容量约束车辆路径问题（CVRP），我们提出了一种贪心启发式算法，旨在生成满足容量约束的巡检路线，并最小化所有故障单车加权完成时间之和。该算法通过迭代构建单趟路线，并根据当前需求动态更新节点，最终逼近最优解。

**贪心算法设计** ：贪心算法从维修点（节点 0）出发，依次选择未访问的节点，直到所有故障单车被收集。每条路线上的容量不得超过 20 单位，算法优先选择需求大的节点，并兼顾行驶时间和搬运时间，以达到减小加权完成时间的目标。

我们的贪心启发式算法主要包括以下步骤：

1. **初始化**
2. **节点选择与路线扩展**：
   - 在累计负荷 $Q^k < C$ 的条件下，遍历所有未访问且需求大于 0 的停车站点：
     - 选择需求 $w_i$ 最大的节点 $i^*$ 作为候选节点。若多个节点需求相等，则选择行驶时间最小的节点
   - 设选中的节点为 $i^*$，收集单车数量为 $\hat{w}_{i^*}^k = \min(w_{i^*}, C - Q^k)$.
   - 更新状态：
     - 累计负荷：$Q^k \leftarrow Q^k + \hat{w}_{i^*}^k$
     - 更新目标函数值：$C_{i^*}^k = C_j^k + T_j^{\text{service}} + T_{j,i^*}^{\text{drive}}$
     - 访问标记：$\text{vis}_{i^*}^k = 1$。
     - 路线更新：将 $i^*$ 加入路线，设置 $y_{j,i^*}^k = 1$，更新当前节点 $j = i^*$
3. **返回维修点**并更新目标函数值
4. **需求更新与多趟迭代**：
   - 每趟结束后，更新节点需求：

$$w_i \leftarrow w_i - \hat{w}_i^k, \quad \forall i \in N.$$

- 累加目标函数值：

$$\sum_{k}\sum_{i=1}^{13} w_i C_i^k.$$

- 若剩余需求大于 0，启动新一趟路线 $k+1$，重复上述步骤；否则终止。

**贪心策略的合理性** ：该策略通过优先选择需求最大的节点，最大化每趟路线的单车收集量，从而减少故障单车的等待时间。次优的距离选择策略有助于优化行驶时间，平衡服务时间的增长。由于问题规模较小（14 个节点），贪心算法能够有效地逼近最优解。

### 7.2 问题四的求解

根据如上的建模，我们运行代码，得到如下的数据：

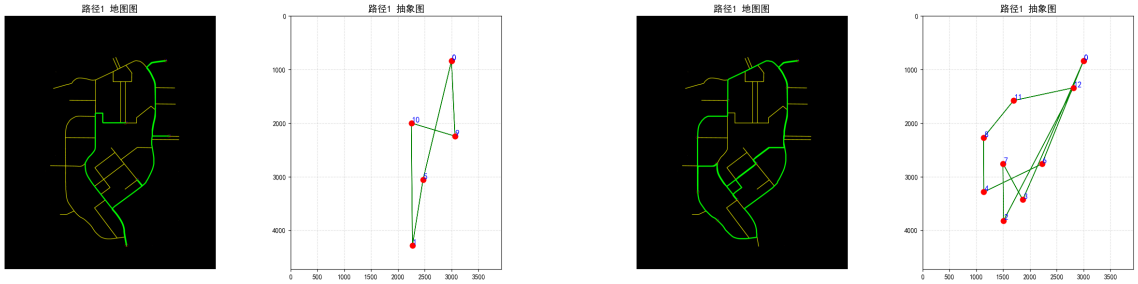
```
Trip 1:
Route: 0 -> 5 -> 1 -> 10 -> 9 -> 0
Collected bikes:
Node 1: 5
Node 5: 6
Node 9: 4
Node 10: 5
Time (min): 35.35
Dis(m): 6395
Demand: 0 3 4 2 0 1 3 2 0 0 2 3

Trip 2:
Route: 0 -> 3 -> 7 -> 2 -> 12 -> 11 -> 8 -> 4 -> 6 -> 0
Collected bikes:
Node 2: 3
Node 3: 4
Node 4: 2
Node 6: 1
Node 7: 3
Node 8: 2
Node 11: 2
Node 12: 3
Time (min): 44.95
Dis(m): 10394
Demand: 0 0 0 0 0 0 0 0 0 0 0 0

Total trips: 2
Total time (min): 80.29
Final demand: 0
```

- **满载效率**：仅需巡检**两趟**，且两趟均恰好装满容量 $C = 20$。
- **时间与距离**：第 1 趟用时 35.35 min，行驶 6395m；第 2 趟用时 44.95min，行驶 10394m，第二趟节点更多耗时和里程较大。
- **需求消除**：第一趟后剩余需求 8 处；第二趟后**全部收集完毕**，需求归零。
- **总耗时**：**80.29 min**，整体路径构造合理，满足“满载即返”策略。

图13分别展示了本模型在两趟收集任务中的实际路径走向。可以看出，算法能够快速聚焦故障单车需求量较大的站点，并在途中动态调整路径选择，充分利用车辆容量，避免了冗余行驶，实现了高效的巡检调度。第一趟（如图13a）主要覆盖需求集中的几个关键节点，而第二趟（如图13b）则在剩余节点之间平衡负荷，整体行程紧凑且逻辑清晰，是一份很合理的行程规划。




(a) 第一次巡检路线　　(b) 第二次巡检路线

**图 13　一天内的巡检路线可视化**

# 八、 模型总结

## 8.1 模型的优点

- **优点 1**：问题一中综合考虑多个因素对单车数量的影响，使用自然三次样条插值的方法来逼近，避免了其他多项式函数逼近会产生的龙格现象，使求得的需求函数更符合实际情况。
- **优点 2**：问题二使用 Dijkstra 算法来求解停车点之间的最短路径的长度，使求解最短路径的时间降低。在建立调度路径评估模型时，考虑最大化运输量以及最小化路径长度，通过二者相减得到综合目标函数，把问题转化为求综合目标函数最大值的问题。同时，在求解的过程中采用遗传算法，通过动态资源竞争和路径耦合机制（协同距离矩阵）提升全局搜索能力。
- **优点 3**：问题三使用了 K-Means 聚类分析和 TOPSIS 优劣解距离法来得到停车点设置的评价模型。本题所用的聚类分析，在原有的的基础上运用了 TOPSIS 的思想，综合考虑两种因素并赋以合适的权。在使用 TOPSIS 的时候，为考虑的五个因素分别赋以合适的权，最终确定最优站点分布。
- **优点 4**：问题四使用了 CVRP 的模型来解决检修共享单车的问题。通过考虑容量上限等因素得到最优解。本题与传统 CVRP 模型不同的是增加了查找和搬运故障车的时间作为考虑因素，引入加权完成时间目标，优先处理高需求节点。同时我们基于此采用了贪心启发式算法以解决多趟容量约束车辆路径问题，增强计算效率。

## 8.2 模型的缺点

- 缺点 1：部分参数的设置受到数据不足的限制，并且高度依赖专家经验。例如，需求函数中空间位置模型的衰减参数，以及问题三中各指标的权重，在实际应用中还需在调整停车点后重新测算和验证其合理性。
- 缺点 2：遗传算法的全局能力受到初始种群和参数设置的影响，容易陷入局部最优解，若是在供需关系和距离矩阵十分复杂的情况下，可能导致结果产生偏差。

## 8.3 模型的改进与推广

为了进一步提升模型在实际场景中的应用效果，我们提出了若干可行的改进方向。首先，可以通过缩短数据采集的时间间隔以获取更高时效性的数据，从而更敏锐地捕捉短时高峰期的共享单车需求，以使模型在调度方案生成过程中提升响应速度与精度。其次，为提高模型在停车点调整后运行效果的可靠性，可以引入实时反馈机制，通过动态监测实际需求变化并及时修正调度策略，能够确保模型在实施阶段与现实环境高度契合。此外，对于模型中涉及

权重设定的步骤，当前多依赖人工经验，存在一定的主观性。为提升决策的科学性和客观性，可引入熵权法等多指标赋权方法，实现对各因素影响程度的合理量化与优化配置。

本模型能够适应不同校园场景下的需求，可以根据具体的运维车辆数、限载、需求情况进行调整，提供不同学校内定制化的解决方案。尽管目前模型的核心应用是针对校园内共享单车系统的优化，但该模型在结构和方法上的灵活性和适应性，使其具备了较强的推广潜力。例如，在城市共享交通系统中，模型的优化思路可推广至其他类型的共享交通工具，通过合理的停放点规划，提升城市交通的运行效率和服务质量。此外，在物流配送、快递站点优化等领域，模型的优化原理也能提供有效的解决方案，推动共享交通和智能城市管理的发展。

## 参考文献


[1] DIJKSTRA E W. A note on two problems in connexion with graphs[J/OL]. Numerische Mathematik, 1959, 1:269-271. DOI: 10.1007/BF01386390.

[2] 陈立华, 梅亚东, 董雅洁, 等. 改进遗传算法及其在水库群优化调度中的应用[J]. 水利学报, 2008(05):550-556.

[3] 谢青成, 毛嘉莉, 刘婷. 城市共享单车的动态调度策略[J]. 华东师范大学学报 (自然科学版), 2019(06):88-102.

[4] 王涵霄, 董明, 张大力. 考虑维修的共享单车调度优化研究[J/OL]. 工业工程与管理, 2019, 24(02):31-37. DOI: 10.19495/j.cnki.1007-5429.2019.02.005.

[5] 陈佳惠, 侯宁, 李晓璐, 等. 共享单车调度路径优化研究[J/OL]. 交通科技与经济, 2021, 23(02):13-20. DOI: 10.19348/j.cnki.issn1008-5696.2021.02.003.

[6] ZHANG W, ZHANG S. Research on multi-objective optimisation for shared bicycle dispatching [J/OL]. International Journal of Vehicle Information and Communication Systems, 2024, 9(4): 372-392. DOI: 10.1504/IJVICS.2024.142094.

[7] 陈萍. 启发式算法及其在车辆路径问题中的应用[D]. 北京: 北京交通大学, 2009.

# 附录 A　所用软件

| 计算工具 | 版本 |
|---|---|
| Python | 3.9.13 |
| MATLAB | R2023a |

# 附录 B　代码

代码均放在支撑文件/code/文件夹下，文件夹内附有说明。

| 文件名 | 语言 | 解决问题 | 说明 |
|---|---|---|---|
| cubic_spline.m | MATLAB | 问题一 | 实现自然三次样条插值 |
| extraction.py | Python | 问题二 | 提取停车位置和黄色道路区域 |
| dijkstra.py | Python | 问题二 | Dijkstra 算法用于最短路径求解 |
| genetic.py | Python | 问题二 | 遗传算法解决共享单车路径规划 |
| topsis_kmeans.m | MATLAB | 问题三 | 带评价准则的 K-means 聚类与 TOPSIS 优化停车点配置 |
| cvrp_greedy.py | Python | 问题四 | CVRP 问题的贪心启发式算法解法 |
| visualization.py | Python | - | 用于图形可视化与数据分析 |

**表 5　程序文件与用途说明**

cubic_spline.m

```
 % 读取Excel文件
[~, ~, raw] = xlsread('附件1-共享单车分布统计表.xlsx'); % 替换为你的文件名

% 初始化存储向量
data_vectors = cell(1, 15); % 存储D-R列数据
time_vectors = cell(1, 15); % 存储对应时间数据

% 列范围 (D到R对应Excel中的4到18列)
start_col = 4;
end_col = 18;

% 先处理时间数据(C列)创建完整time_vector
time_vector = zeros(34, 1); % 预分配空间
for row = 2:35
    if size(raw,1) >= row && size(raw,2) >= 3
        time_content = raw{row, 3};
        total_hours = 0;

        if ischar(time_content)
            time_parts = strsplit(time_content, ':');
            if ~isempty(time_parts)
                hours = str2double(time_parts{1});
                if ~isnan(hours), total_hours = hours; end

```

```
                if length(time_parts) >= 2
                    minutes = str2double(time_parts{2});
                    if ~isnan(minutes), total_hours = total_hours + minutes/60; end
                end

                if length(time_parts) >= 3
                    seconds = str2double(time_parts{3});
                    if ~isnan(seconds), total_hours = total_hours + seconds/3600; end
                end
            end
        elseif isnumeric(time_content)
            total_hours = time_content;
        end

        % 添加行偏移
        if row >= 2 && row <= 9
            % 不加
        elseif row >= 10 && row <= 17
            total_hours = total_hours + 1;
        elseif row >= 18 && row <= 25
            total_hours = total_hours + 2;
        elseif row >= 26 && row <= 30
            total_hours = total_hours + 3;
        elseif row >= 31 && row <= 35
            total_hours = total_hours + 4;
        end

        time_vector(row-1) = round(total_hours, 5);
    end
end

% 处理数据列并创建对应时间向量
for col = start_col:end_col
    current_data = [];
    current_time = [];

    for row = 2:35
        if size(raw,1) >= row && size(raw,2) >= col
            cell_content = raw{row, col};
            valid_data = false;

            if isnumeric(cell_content) && ~isnan(cell_content)
                current_data = [current_data; cell_content];
                valid_data = true;
            elseif ischar(cell_content)
                if strcmp(strtrim(cell_content), '200+')
                    current_data = [current_data; 200];
                    valid_data = true;
                else
                    num = str2double(cell_content);
                    if ~isnan(num)
                        current_data = [current_data; num];
                        valid_data = true;
                    end
                end
            end

            % 只有当数据有效时才添加对应时间
            if valid_data
                current_time = [current_time; time_vector(row-1)];
            end
        end
    end

    % 在数据向量前后插入60
    current_data = [60; current_data; 60];

    % 在时间向量前后插入0和5
    current_time = [0; current_time; 5];

    % 存储向量
    data_vectors{col-start_col+1} = current_data;
    time_vectors{col-start_col+1} = current_time;
```

```
end

% 分配到工作区
for i = 1:15
    col_char = char('C' + i); % D=4→'D',...,R=18→'R'
    assignin('base', ['vector_' col_char], data_vectors{i});
    assignin('base', ['time_vector_' col_char], time_vectors{i});
end

% 显示结果
disp('数据提取完成:');
disp(['1. 数据向量已存储为vector_D到vector_R (' num2str(length(data_vectors)) '个向量)']);
disp('   每个数据向量开头和结尾已插入60');
disp(['2. 对应时间向量已存储为time_vector_D到time_vector_R (' num2str(length(time_vectors)) '个向量)']);
disp('   每个时间向量开头已插入0，结尾已插入5');
disp('对应关系说明:');
disp('  - 插入后，每个数据向量中的第2到end-1个元素对应时间向量中的第2到end-1个元素');
disp('  - 原始对应关系保持不变，只是前后各添加了一个固定值');

% 做自然三次样条插值
ppD = csape(time_vector_D,vector_D,'variational');
ppE = csape(time_vector_E,vector_E,'variational');
ppF = csape(time_vector_F,vector_F,'variational');
ppG = csape(time_vector_G,vector_G,'variational');
ppH = csape(time_vector_H,vector_H,'variational');
ppI = csape(time_vector_I,vector_I,'variational');
ppJ = csape(time_vector_J,vector_J,'variational');
ppK = csape(time_vector_K,vector_K,'variational');
ppL = csape(time_vector_L,vector_L,'variational');
ppM = csape(time_vector_M,vector_M,'variational');
ppN = csape(time_vector_N,vector_N,'variational');
ppO = csape(time_vector_O,vector_O,'variational');
ppP = csape(time_vector_P,vector_P,'variational');
ppQ = csape(time_vector_Q,vector_Q,'variational');
ppR = csape(time_vector_R,vector_R,'variational');

% 设置时间坐标
time1 = [7 9 12 14 18 21 23]/24;
time2 = [7 9 12 14 18 21 23]/24+1;
time3 = [7 9 12 14 18 21 23]/24+2;
time4 = [7 9 12 14 18 21 23]/24+3;
time5 = [7 9 12 14 18 21 23]/24+4;

% 带入插值函数值求平均
bike_D = (fnval(ppD,time1)+fnval(ppD,time2)+fnval(ppD,time3)+fnval(ppD,time4)+fnval(ppD,time5))/5;
bike_E = (fnval(ppE,time1)+fnval(ppE,time2)+fnval(ppE,time3)+fnval(ppE,time4)+fnval(ppE,time5))/5;
bike_F = (fnval(ppF,time1)+fnval(ppF,time2)+fnval(ppF,time3)+fnval(ppF,time4)+fnval(ppF,time5))/5;
bike_G = (fnval(ppG,time1)+fnval(ppG,time2)+fnval(ppG,time3)+fnval(ppG,time4)+fnval(ppG,time5))/5;
bike_H = (fnval(ppH,time1)+fnval(ppH,time2)+fnval(ppH,time3)+fnval(ppH,time4)+fnval(ppH,time5))/5;
bike_I = (fnval(ppI,time1)+fnval(ppI,time2)+fnval(ppI,time3)+fnval(ppI,time4)+fnval(ppI,time5))/5;
bike_J = (fnval(ppJ,time1)+fnval(ppJ,time2)+fnval(ppJ,time3)+fnval(ppJ,time4)+fnval(ppJ,time5))/5;
bike_K = (fnval(ppK,time1)+fnval(ppK,time2)+fnval(ppK,time3)+fnval(ppK,time4)+fnval(ppK,time5))/5;
bike_L = (fnval(ppL,time1)+fnval(ppL,time2)+fnval(ppL,time3)+fnval(ppL,time4)+fnval(ppL,time5))/5;
bike_M = (fnval(ppM,time1)+fnval(ppM,time2)+fnval(ppM,time3)+fnval(ppM,time4)+fnval(ppM,time5))/5;
bike_N = (fnval(ppN,time1)+fnval(ppN,time2)+fnval(ppN,time3)+fnval(ppN,time4)+fnval(ppN,time5))/5;
bike_O = (fnval(ppO,time1)+fnval(ppO,time2)+fnval(ppO,time3)+fnval(ppO,time4)+fnval(ppO,time5))/5;
bike_P = (fnval(ppP,time1)+fnval(ppP,time2)+fnval(ppP,time3)+fnval(ppP,time4)+fnval(ppP,time5))/5;
bike_Q = (fnval(ppQ,time1)+fnval(ppQ,time2)+fnval(ppQ,time3)+fnval(ppQ,time4)+fnval(ppQ,time5))/5;
bike_R = (fnval(ppR,time1)+fnval(ppR,time2)+fnval(ppR,time3)+fnval(ppR,time4)+fnval(ppR,time5))/5;
```

extraction.py

```
import cv2
import numpy as np

# 路径和输出文件
map_path = "map.png"
output_parking_path = "roads_with_parking.png"
output_parking_coords = "parking_location.txt"
output_yellow_path = "yellow_roads.png"  # 黄色道路图像路径

# 读取地图图像
img = cv2.imread(map_path)
```

```python
if img is None:
    print("地图图像加载失败！")
    exit()

# 提取黄色道路
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
yellow_lower = np.array([20, 100, 100])  # 黄色范围下界
yellow_upper = np.array([30, 255, 255])  # 黄色范围上界
yellow_mask = cv2.inRange(hsv, yellow_lower, yellow_upper)
yellow_img = cv2.bitwise_and(img, img, mask=yellow_mask)

# 保存黄色道路图像
cv2.imwrite(output_yellow_path, yellow_img)

# 转HSV提取红色停车点
red_mask = cv2.inRange(img, (0, 0, 255), (0, 0, 255))  # 精确匹配红色停车点
contours, _ = cv2.findContours(red_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 获取停车点坐标
parking_points = []
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area < 100:
        continue

    x, y, w, h = cv2.boundingRect(cnt)
    parking_points.append((x + w // 2, y + h + 10 // 2))  # 停车点坐标中心

# 寻找最接近的黄色道路点并更新停车点位置
def find_nearest_yellow_point(parking_point, yellow_binary, max_radius=50):
    x_start, y_start = parking_point
    for radius in range(1, max_radius + 1):
        y_min = max(y_start - radius, 0)
        y_max = min(y_start + radius, yellow_binary.shape[0])
        x_min = max(x_start - radius, 0)
        x_max = min(x_start + radius, yellow_binary.shape[1])
        sub_region = yellow_binary[y_min:y_max, x_min:x_max]
        coords = np.column_stack(np.where(sub_region == 255))
        if len(coords) > 0:
            nearest_yellow_point = coords[np.argmin(np.linalg.norm(coords - [radius, radius], axis=1))]
            nearest_x = x_min + nearest_yellow_point[1]
            nearest_y = y_min + nearest_yellow_point[0]
            return (nearest_x, nearest_y)
    return parking_point

# 生成黄色道路二值图
yellow_binary = cv2.cvtColor(yellow_img, cv2.COLOR_BGR2GRAY)
_, yellow_binary = cv2.threshold(yellow_binary, 1, 255, cv2.THRESH_BINARY)

# 更新停车点位置
updated_parking_points = [find_nearest_yellow_point(point, yellow_binary) for point in parking_points]
updated_parking_points[0] = (2273,4295)  # 可手动调整特定停车点

# 输出停车点图像
output_img = yellow_img.copy()
for x, y in updated_parking_points:
    cv2.circle(output_img, (x, y), 20, (0, 0, 255), -1)  # 红色圆圈标记停车点

# 保存合成图像
cv2.imwrite(output_parking_path, output_img)

# 保存停车点坐标到文件
with open(output_parking_coords, 'w') as f:
    for point in updated_parking_points:
        f.write(f"{point[0]}, {point[1]}\n")

# 可视化结果
cv2.imshow("Yellow Roads + Parking Points", output_img)
cv2.waitKey(0)
cv2.destroyAllWindows()

print(f"黄色道路已保存为 {output_yellow_path}")
print(f"停车点和黄色道路图像已保存为 {output_parking_path}")
```

```python
print(f"停车点坐标已保存到 {output_parking_coords}")
```

dijkstra.py

```python
import cv2
import numpy as np
import os
import heapq
from concurrent.futures import ProcessPoolExecutor
from tqdm import tqdm  # 添加进度条

# 路径配置
yellow_path = "yellow_roads.png"
parking_location_file = "parking_location.txt"
output_distance_with_meters_file = "distances_with_meters.txt"

# 图像和坐标点
yellow_img = cv2.imread(yellow_path)
with open(parking_location_file, 'r') as f:
    parking_points = [tuple(map(int, line.strip().split(", "))) for line in f]

if yellow_img is None:
    print("黄色道路图像加载失败！")
    exit()

yellow_binary = cv2.cvtColor(yellow_img, cv2.COLOR_BGR2GRAY)
_, yellow_binary = cv2.threshold(yellow_binary, 1, 255, cv2.THRESH_BINARY)

pixel_to_meter = 2000 / 3770
directions = [(-1, 0, 1.0), (1, 0, 1.0), (0, -1, 1.0), (0, 1, 1.0),
              (-1, -1, 1.4142), (-1, 1, 1.4142), (1, -1, 1.4142), (1, 1, 1.4142)]


def dijkstra_all(start_idx):
    start = parking_points[start_idx]
    h, w = yellow_binary.shape
    dist = np.full((h, w), np.inf, dtype=np.float32)
    dist[start[1], start[0]] = 0.0
    heap = [(0.0, start)]

    while heap:
        cur_dist, (x, y) = heapq.heappop(heap)
        for dx, dy, cost in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < w and 0 <= ny < h and yellow_binary[ny, nx] == 255:
                new_dist = cur_dist + cost
                if new_dist < dist[ny, nx]:
                    dist[ny, nx] = new_dist
                    heapq.heappush(heap, (new_dist, (nx, ny)))

    results = []
    for j in range(len(parking_points)):
        if j == start_idx:
            continue
        end = parking_points[j]
        d = dist[end[1], end[0]]
        if not np.isinf(d):
            meter = d * pixel_to_meter
            results.append((start_idx, j, d, meter))
    return results


if __name__ == "__main__":
    print("开始并行计算所有最短路径（带进度条）...")
    all_results = []
    with ProcessPoolExecutor() as executor:
        for res_list in tqdm(executor.map(dijkstra_all, range(len(parking_points))), total=len(parking_points)):
            all_results.append(res_list)

    with open(output_distance_with_meters_file, 'w') as f:
        for res_list in all_results:
            for i, j, d, meter in res_list:
```

```
                print(f"记录: {i} -> {j} = {d:.2f} 像素")
                f.write(f"{i} {j} {d:.4f} {meter:.2f}\n")
```

genetic.py

```
import random
import time
import copy
import math

NUM_STATIONS = 15
NUM_VEHICLES = 3
MAX_CAPACITY = 20
MAX_STEPS = 13
POP_SIZE = 20
MAX_GENERATIONS = 8000
MAX_PATH_DISTANCE = 416 * 60

vehicle_balance = [14, 36, 8, -16, -59, -10, -29, -7, 41, -27, 14, 24, -2, 2, 3]

distance_matrix = [
    [0, 599, 852, 465, 985, 949, 1020, 1248, 1516, 1622, 1521, 2072, 1789, 2180, 2239],
    [599, 0, 241, 872, 377, 1354, 845, 1611, 1927, 1039, 1328, 1897, 2187, 2001, 2064],
    [852, 241, 0, 1123, 360, 1606, 822, 1591, 2118, 1017, 1313, 1877, 2272, 1990, 2034],
    [465, 872, 1123, 0, 951, 548, 550, 777, 1119, 1148, 1053, 1597, 1392, 1711, 1762],
    [985, 377, 360, 951, 0, 1334, 462, 1224, 1756, 662, 948, 1517, 1908, 1621, 1678],
    [949, 1354, 1606, 548, 1334, 0, 929, 410, 819, 1541, 1434, 1599, 1085, 2099, 1807],
    [1020, 845, 822, 550, 462, 929, 0, 821, 1356, 662, 501, 1053, 1445, 1154, 1208],
    [1248, 1611, 1591, 777, 1224, 410, 821, 0, 714, 1422, 1326, 1498, 983, 1983, 1711],
    [1516, 1927, 2118, 1119, 1756, 819, 1356, 714, 0, 1631, 1342, 1125, 611, 1876, 1341],
    [1622, 1039, 1017, 1148, 662, 1541, 662, 1422, 1631, 0, 285, 874, 1270, 986, 1037],
    [1521, 1328, 1313, 1053, 948, 1434, 501, 1326, 1342, 285, 0, 589, 981, 698, 753],
    [2072, 1897, 1877, 1597, 1517, 1599, 1053, 1498, 1125, 874, 589, 0, 769, 912, 553],
    [1789, 2187, 2272, 1392, 1908, 1085, 1445, 983, 611, 1270, 981, 769, 0, 1253, 721],
    [2180, 2001, 1990, 1711, 1621, 2099, 1154, 1983, 1876, 986, 698, 912, 1253, 0, 682],
    [2239, 2064, 2034, 1762, 1678, 1807, 1208, 1711, 1341, 1037, 753, 553, 721, 682, 0],
]

def min3(a, b, c):
    return min(a, min(b, c))

def generate_random_schedule():
    schedule = []
    for _ in range(NUM_VEHICLES):
        steps = []
        first_station = next((i for i, v in enumerate(vehicle_balance) if v > 0), None)
        if first_station is None:
            schedule.append([])
            continue
        m = 1 + random.randint(0, MAX_STEPS - 1)
        first_load = min(vehicle_balance[first_station], MAX_CAPACITY)
        steps.append((first_station, -first_load))
        last_station = first_station
        for _ in range(1, m):
            demand_station = max(((i, abs(v)) for i, v in enumerate(vehicle_balance) if v < 0 and i !=
    last_station),
                                 key=lambda x: x[1], default=(None, 0))[0]
            if demand_station is not None:
                station = demand_station
            else:
                station = random.choice([i for i in range(NUM_STATIONS) if i != last_station])
            last_station = station
            op = 1 if random.randint(0, 1) == 0 else -1
            load_change = op * (1 + random.randint(0, 19))
            load_change = max(-20, min(20, load_change))
            steps.append((station, load_change))
        schedule.append(steps)
    return schedule

def calculate_fitness(schedule):
    balance_copy = vehicle_balance[:]
    total_distance = 0
    vehicle_positions = [0] * NUM_VEHICLES
```

```
    vehicle_loads = [0] * NUM_VEHICLES
    vehicle_last_station = [-1] * NUM_VEHICLES
    all_finished = False
    while not all_finished:
        all_finished = True
        for v in range(NUM_VEHICLES):
            if vehicle_positions[v] >= len(schedule[v]):
                continue
            all_finished = False
            station, plan_delta = schedule[v][vehicle_positions[v]]
            if vehicle_last_station[v] != -1:
                total_distance += distance_matrix[vehicle_last_station[v]][station]
            vehicle_last_station[v] = station
            current_balance = balance_copy[station]
            current_load = vehicle_loads[v]
            actual_delta = 0
            if plan_delta > 0 and current_balance < 0:
                actual_delta = min3(plan_delta, -current_balance, current_load)
            elif plan_delta < 0 and current_balance > 0:
                actual_delta = -min(-plan_delta, min(current_balance, MAX_CAPACITY - current_load))
            balance_copy[station] += actual_delta
            vehicle_loads[v] -= actual_delta
            if vehicle_loads[v] < 0 or vehicle_loads[v] > MAX_CAPACITY:
                return 1e18
            vehicle_positions[v] += 1
    if total_distance > MAX_PATH_DISTANCE:
        return 1e18
    M = sum(200 * max(-b, 0)**2 + max(b, 0) for b in balance_copy)
    return M + total_distance

def calculate_M(balance):
    return sum((max(-b, 0) ** 2 + max(b, 0)) for b in balance)

def get_m_i(balance_i):
    return max(-balance_i, 0)**2 + max(balance_i, 0)

def get_actual_deltas(schedule):
    actual_deltas = [[0]*MAX_STEPS for _ in range(NUM_VEHICLES)]
    balance_copy = vehicle_balance[:]
    vehicle_positions = [0] * NUM_VEHICLES
    vehicle_loads = [0] * NUM_VEHICLES
    all_finished = False
    while not all_finished:
        all_finished = True
        for v in range(NUM_VEHICLES):
            if vehicle_positions[v] >= len(schedule[v]):
                continue
            all_finished = False
            station, plan_delta = schedule[v][vehicle_positions[v]]
            current_balance = balance_copy[station]
            current_load = vehicle_loads[v]
            actual_delta = 0
            if plan_delta > 0 and current_balance < 0:
                actual_delta = min3(plan_delta, -current_balance, current_load)
            elif plan_delta < 0 and current_balance > 0:
                actual_delta = -min(-plan_delta, min(current_balance, MAX_CAPACITY - current_load))
            actual_deltas[v][vehicle_positions[v]] = actual_delta
            balance_copy[station] += actual_delta
            vehicle_loads[v] -= actual_delta
            vehicle_positions[v] += 1
    return actual_deltas

def get_final_balance(schedule):
    balance_copy = vehicle_balance[:]
    vehicle_positions = [0] * NUM_VEHICLES
    vehicle_loads = [0] * NUM_VEHICLES
    all_finished = False
    while not all_finished:
        all_finished = True
        for v in range(NUM_VEHICLES):
            if vehicle_positions[v] >= len(schedule[v]):
                continue
            all_finished = False
```

```python
            station, plan_delta = schedule[v][vehicle_positions[v]]
            current_balance = balance_copy[station]
            current_load = vehicle_loads[v]
            actual_delta = 0
            if plan_delta > 0 and current_balance < 0:
                actual_delta = min3(plan_delta, -current_balance, current_load)
            elif plan_delta < 0 and current_balance > 0:
                actual_delta = -min(-plan_delta, min(current_balance, MAX_CAPACITY - current_load))
            balance_copy[station] += actual_delta
            vehicle_loads[v] -= actual_delta
            vehicle_positions[v] += 1
    return balance_copy

def main():
    random.seed(time.time())
    population = [generate_random_schedule() for _ in range(POP_SIZE)]
    fitness = [calculate_fitness(ind) for ind in population]
    best_idx = min(range(POP_SIZE), key=lambda i: fitness[i])
    best = copy.deepcopy(population[best_idx])
    best_fitness = fitness[best_idx]
    for _ in range(MAX_GENERATIONS):
        new_population = []
        for _ in range(POP_SIZE):
            p1, p2 = random.sample(range(POP_SIZE), 2)
            parent1 = population[p1] if fitness[p1] < fitness[p2] else population[p2]
            q1, q2 = random.sample(range(POP_SIZE), 2)
            parent2 = population[q1] if fitness[q1] < fitness[q2] else population[q2]
            offspring = [random.choice([v1, v2]) for v1, v2 in zip(parent1, parent2)]
            for v in range(NUM_VEHICLES):
                for s in range(len(offspring[v])):
                    if random.randint(0, 9) == 0:
                        offspring[v][s] = (random.randint(0, NUM_STATIONS - 1), offspring[v][s][1])
                    if random.randint(0, 9) == 0:
                        offspring[v][s] = (offspring[v][s][0], random.randint(-20, 20))
            new_population.append(offspring)
        population = new_population
        fitness = [calculate_fitness(ind) for ind in population]
        gen_best_idx = min(range(POP_SIZE), key=lambda i: fitness[i])
        if fitness[gen_best_idx] < best_fitness:
            best_fitness = fitness[gen_best_idx]
            best = copy.deepcopy(population[gen_best_idx])

    print(f"Best Fitness: {best_fitness}")
    actual_deltas = get_actual_deltas(best)
    for v in range(NUM_VEHICLES):
        print(f"Vehicle {v}:")
        load = 0
        last_station = -1
        dist = 0
        for s, (station, _) in enumerate(best[v]):
            if last_station != -1:
                dist += distance_matrix[last_station][station]
            last_station = station
            load += -actual_deltas[v][s]
            print(f"  Station {station}: {actual_deltas[v][s]:+d} cars (current load: {load})")
        print(f"  Total distance: {dist}")
    final_balance = get_final_balance(best)
    M_before = calculate_M(vehicle_balance)
    M_after = calculate_M(final_balance)
    efficiency = 100 - (M_after / M_before * 100)
    print(f"Efficiency: {efficiency:.2f}%")
    print("Station-wise metrics and final balance:")
    for i in range(NUM_STATIONS):
        print(f"Station {i}: {vehicle_balance[i]:+d} -> {final_balance[i]:+d}")

if __name__ == '__main__':
    main()
```

topsis_kmeans.m

```python
% ------------------------------
% 阶段一：带评价准则的 K-means 聚类
% ------------------------------
```

```

[row, col, count] = find(distribution);  % 非零点坐标与单车数量，在问题二中得到
load('important_locations.mat');%在问题二中得到
parking_points = [col, row];  % X=col, Y=row
bike_counts = count;

% ---------- 评价准则 ----------
predicted_counts = 0.9 * bike_counts;
supply_demand_match = 1 - abs(predicted_counts - bike_counts) ./ max(predicted_counts, bike_counts);

avg_distance = zeros(size(parking_points,1), 1);
for i = 1:size(parking_points, 1)
    d = sqrt(sum((important_locations - parking_points(i,:)).^2, 2));
    avg_distance(i) = mean(d);
end

% ---------- 多 K 聚类 ----------
K_range = 8:15;
all_results = struct();  % 保存所有K的聚类结果
max_sample = 2000;       % 每次最多采样这么多点

for K = K_range
    fprintf('处理 K = %d...\n', K);

    % 如果数据量太大就采样
    N = size(parking_points, 1);
    sample_idx = randsample(N, min(max_sample, N));
    sampled_points = parking_points(sample_idx, :);
    sampled_sdm = supply_demand_match(sample_idx);
    sampled_dist = avg_distance(sample_idx);

    best_J = Inf;
    best_idx = [];
    best_C = [];

    % 多次聚类尝试
    for trial = 1:5
        try
            [idx, C] = kmeans(sampled_points, K, ...
                'Start', 'plus', ...
                'MaxIter', 300, ...
                'Replicates', 1, ...
                'Display', 'off');

            % 计算目标函数
            cluster_SSE = 0;
            cluster_S3 = 0;
            for k = 1:K
                members = (idx == k);
                if sum(members) == 0
                    continue;
                end
                cluster_SSE = cluster_SSE + mean(sampled_sdm(members));
                cluster_S3 = cluster_S3 + mean(sampled_dist(members));
            end

            cluster_SSE = cluster_SSE / K;
            cluster_S3 = cluster_S3 / K;
            J = -0.8 * cluster_SSE + 0.2 * cluster_S3;

            if J < best_J
                best_J = J;
                best_idx = idx;
                best_C = C;
            end
        catch ME
            warning('K-means 聚类失败: %s', ME.message);
        end
    end

    % 保存结果
    all_results(K).K = K;
    all_results(K).idx = best_idx;
```

```
    all_results(K).centroids = best_C;
    all_results(K).J = best_J;
    all_results(K).sampled_points = sampled_points;

    % 可视化
    figure;
    gscatter(sampled_points(:,1), sampled_points(:,2), best_idx);
    hold on;
    plot(best_C(:,1), best_C(:,2), 'kx', 'MarkerSize', 12, 'LineWidth', 2);
    title(['K = ', num2str(K), ' 的最优聚类']);
    xlabel('X');
    ylabel('Y');
    legend('Location', 'bestoutside');
    hold off;
end

% ----------- 阶段二: TOPSIS 评估各K聚类方案 -----------
% 权重: 与LaTeX中表格一致
weights = [0.06, 0.04, 0.13, 0.42, 0.35];  % [S1, S2, S3, S4, S5]

% 初始化指标矩阵, 行数 = K个方案, 列数 = 5个指标
num_K = length(K_range);
criteria_matrix = zeros(num_K, 5);

k_idx = 1;
for K = K_range
    result = all_results(K);
    C = result.centroids;
    idx = result.idx;
    points = result.sampled_points;

    % --- S1: 聚类数量 ---
    S1 = K;

    % --- S2: 聚类中心之间的平均距离 ---
    pairwise_dist = pdist(C);
    S2 = mean(pairwise_dist);

    % --- S3: 聚类中心到重要位置的平均距离 ---
    center_to_important = zeros(K, 1);
    for i = 1:K
        dists = sqrt(sum((important_locations - C(i,:)).^2, 2));
        center_to_important(i) = mean(dists);
    end
    S3 = mean(center_to_important);

    % --- S4: 供需匹配率 (聚类内部平均) ---
    sdm_vals = supply_demand_match;
    sampled_idx = ismember(parking_points, points, 'rows');  % 匹配供需数据
    cluster_sdm = 0;
    for k = 1:K
        members = (idx == k);
        cluster_points = points(members,:);
        match_idx = ismember(parking_points, cluster_points, 'rows');
        cluster_sdm = cluster_sdm + mean(sdm_vals(match_idx));
    end
    S4 = cluster_sdm / K;

    % --- S5: 调度成本 (各点到对应中心的距离 × 点数) ---
    total_cost = 0;
    for k = 1:K
        members = (idx == k);
        cluster_points = points(members,:);
        center = C(k,:);
        dists = sqrt(sum((cluster_points - center).^2, 2));
        total_cost = total_cost + sum(dists);
    end
    S5 = total_cost;

    % 填入评价矩阵
    criteria_matrix(k_idx, :) = [S1, S2, S3, S4, S5];
    k_idx = k_idx + 1;
end
```

```

% ----------- TOPSIS 处理 -----------
% 1. 极小化指标：S1, S2, S3, S5 → 越小越好；S4 越大越好
% 将 S4 转换为负向（用于统一最小化处理）
criteria_matrix(:,4) = -criteria_matrix(:,4);  % S4反转

% 2. 归一化
norm_matrix = criteria_matrix ./ vecnorm(criteria_matrix, 2, 1);

% 3. 加权
weighted_matrix = norm_matrix .* weights;

% 4. 确定理想解与负理想解
positive_ideal = min(weighted_matrix);  % 越小越好
negative_ideal = max(weighted_matrix);

% 5. 计算与理想解的距离
D_plus = sqrt(sum((weighted_matrix - positive_ideal).^2, 2));
D_minus = sqrt(sum((weighted_matrix - negative_ideal).^2, 2));

% 6. 得分
scores = D_minus ./ (D_plus + D_minus);

% 找出得分最高的 K
[~, best_k_idx] = max(scores);  % 使用 best_k_idx 替代 max_idx
best_K_value = K_range(best_k_idx);

% 输出最优聚类数K和得分
fprintf('TOPSIS 最优聚类数 K = %d，得分 = %.4f\n', best_K_value, scores(best_k_idx));

% 可视化 TOPSIS 结果
figure;
plot(K_range, scores, '-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('聚类数 K');
ylabel('TOPSIS 得分');
title('不同 K 值的 TOPSIS 得分');
grid on;

% 读取地图
map_img = imread('map.png');

% 获取道路像素位置（黄色）
yellow_mask = map_img(:,:,1) == 255 & map_img(:,:,2) == 255 & map_img(:,:,3) == 0;
[road_y, road_x] = find(yellow_mask);
road_pixels = [road_x, road_y];


best_centroids = all_results(best_K_value).centroids;

% 对每个聚类中心，找到最近的道路点
parking_locations = zeros(size(best_centroids));
for i = 1:size(best_centroids, 1)
    center = best_centroids(i, :);
    distances = sum((road_pixels - center).^2, 2);
    [~, min_idx] = min(distances);
    parking_locations(i, :) = road_pixels(min_idx, :);
end

disp('选定的道路停车点坐标：');
disp(parking_locations);
save('final_parking_locations.mat', 'parking_locations');

figure;
imshow(map_img);
hold on;
plot(parking_locations(:,1), parking_locations(:,2), 'ro', 'MarkerSize', 8, 'LineWidth', 2);
title('投影到道路上的停车点');
legend('最终停车点');
hold off;
```

cvrp_greedy.py

```
import math
```

```python

INF = float('inf')
SPEED = 25.0 * 1000.0 / 60.0  # m/min
LOAD_TIME = 1.0               # min per unit load
N = 20

# Demand and Distance
n = 13
cap = 20
demand = [0, 5, 3, 4, 2, 6, 1, 3, 2, 4, 5, 2, 3] + [0]*(N-13)

# Static distance matrix (truncated for readability)
dist = [
    [0, 2321, 2537, 2020, 2162, 1593, 1487, 1700, 1647, 1121, 1085, 1028, 455, 615],
    [2321, 0, 573, 543, 952, 900, 1102, 984, 1472, 1543, 1922, 1651, 1868, 2229],
    [2537, 573, 0, 934, 380, 1299, 1494, 840, 922, 1942, 1780, 1509, 2267, 2087],
    [2020, 543, 934, 0, 904, 599, 561, 442, 930, 1243, 1380, 1110, 1567, 1688],
    [2162, 952, 380, 904, 0, 1325, 1119, 465, 546, 1825, 1404, 1134, 2035, 1712],
    [1593, 900, 1299, 599, 1325, 0, 213, 863, 1351, 815, 1330, 1531, 1140, 1778],
    [1487, 1102, 1494, 561, 1119, 213, 0, 657, 1145, 709, 1224, 1324, 1034, 1672],
    [1700, 984, 840, 442, 465, 863, 657, 0, 491, 1363, 943, 672, 1573, 1250],
    [1647, 1472, 922, 930, 546, 1351, 1145, 491, 0, 1746, 889, 619, 1520, 1197],
    [1121, 1543, 1942, 1243, 1825, 815, 709, 1363, 1746, 0, 859, 1376, 669, 1307],
    [1085, 1922, 1780, 1380, 1404, 1330, 1224, 943, 889, 859, 0, 519, 633, 798],
    [1028, 1651, 1509, 1110, 1134, 1531, 1324, 672, 619, 1376, 519, 0, 1054, 578],
    [455, 1868, 2267, 1567, 2035, 1140, 1034, 1573, 1520, 669, 633, 1054, 0, 641],
    [615, 2229, 2087, 1688, 1712, 1778, 1672, 1250, 1197, 1307, 798, 578, 641, 0]
]


def trip():
    vis = [False] * N
    collected = [0] * N
    route = [-1] * N
    cur = 0
    cur_cap = 0
    length = 0
    travel = 0.0
    load_time = 0.0
    vis[0] = True
    route[length] = 0
    length += 1

    while cur_cap < cap:
        nxt = -1
        max_demand = -1
        min_dist = INF
        for i in range(1, n):
            if not vis[i] and demand[i] > 0:
                if demand[i] > max_demand or (demand[i] == max_demand and dist[cur][i] < min_dist):
                    max_demand = demand[i]
                    min_dist = dist[cur][i]
                    nxt = i
        if nxt == -1:
            break

        collect = min(demand[nxt], cap - cur_cap)
        collected[nxt] = collect
        cur_cap += collect
        travel += dist[cur][nxt] / SPEED
        load_time += collect * LOAD_TIME
        vis[nxt] = True
        route[length] = nxt
        length += 1
        cur = nxt

    if cur != 0:
        travel += dist[cur][0] / SPEED
        route[length] = 0
        length += 1

    return route[:length], collected, travel + load_time

```

```python

def main():
    trips = 0
    total_time = 0.0

    while sum(demand[1:n]) > 0:
        trips += 1
        route, collected, t = trip()
        print(f"\nTrip {trips}:")
        print("Route:", " -> ".join(str(node) for node in route))
        print("Collected bikes:")
        for i in range(1, n):
            if collected[i] > 0:
                print(f"Node {i}: {collected[i]}")
                demand[i] -= collected[i]
        dist_sum = sum(dist[route[i]][route[i+1]] for i in range(len(route)-1))
        print(f"Time (min): {t:.2f}")
        print(f"Dis(m): {dist_sum}")
        print("Demand:", " ".join(map(str, demand[1:n])))
        total_time += t

    print(f"\nTotal trips: {trips}")
    print(f"Total time (min): {total_time:.2f}")
    print("Final demand:", " ".join(map(str, demand[1:n])))


if __name__ == '__main__':
    main()
```

visualization.py

```python
import cv2
import numpy as np
import os
from collections import deque
import matplotlib.pyplot as plt
import matplotlib

# 设置中文和负号正常显示
matplotlib.rcParams['font.sans-serif'] = ['SimHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# 路径文件
yellow_path = "extracted_yellow_roads_1.png"
parking_location_file = "parking_location.txt"

# 路径数组
path1 = [0, 3, 7, 2, 12, 11, 8, 4, 6, 0]                # Vehicle 0 的访问站点顺序

# 读取图像与停车点
yellow_img = cv2.imread(yellow_path)
with open(parking_location_file, 'r') as f:
    parking_points = [tuple(map(int, line.strip().split(", "))) for line in f]

if yellow_img is None:
    print("黄色道路图像加载失败！")
    exit()

yellow_binary = cv2.cvtColor(yellow_img, cv2.COLOR_BGR2GRAY)
_, yellow_binary = cv2.threshold(yellow_binary, 1, 255, cv2.THRESH_BINARY)

# BFS 路径函数
def bfs_find_path(start, end, binary_img):
    queue = deque([start])
    visited = np.zeros_like(binary_img, dtype=np.uint8)
    visited[start[1], start[0]] = 1
    parent = {}
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)]

    while queue:
        current = queue.popleft()
        x, y = current

```

```python
        if current == end:
            path = []
            while current != start:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()
            return path

        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < binary_img.shape[1] and 0 <= ny < binary_img.shape[0]:
                if binary_img[ny, nx] == 255 and not visited[ny, nx]:
                    queue.append((nx, ny))
                    visited[ny, nx] = 1
                    parent[(nx, ny)] = (x, y)

    return []

# 显示单条路径的两个图（返回原图和抽象图）
def render_path_pair(path, title_prefix):
    # 创建图像副本
    img_copy = yellow_img.copy()

    # BFS 路径绘制
    for idx in range(len(path) - 1):
        start_idx = path[idx]
        end_idx = path[idx + 1]
        start_point = parking_points[start_idx]
        end_point = parking_points[end_idx]

        path_points = bfs_find_path(start_point, end_point, yellow_binary)
        for x, y in path_points:
            cv2.circle(img_copy, (x, y), 10, (0, 255, 0), -1)

    # 标注编号点
    for index in path:
        pt = parking_points[index]
        cv2.circle(img_copy, pt, 30, (0, 0, 255), 2)
        cv2.putText(img_copy, str(index), (pt[0]+10, pt[1]-10), cv2.FONT_HERSHEY_SIMPLEX,
                    0.8, (255, 0, 0), 2, cv2.LINE_AA)

    # 原图显示对象（BGR转RGB）
    map_view = cv2.cvtColor(img_copy, cv2.COLOR_BGR2RGB)

    # 抽象图数据准备
    abstract_fig = {
        "points": [parking_points[i] for i in path],
        "labels": path,
        "title": f"{title_prefix} 抽象图"
    }

    return map_view, abstract_fig

# 只选择一条路径
path = path1

# 渲染单条路径
map_img, abs_data = render_path_pair(path, "路径1")

# 绘图
fig, axs = plt.subplots(1, 2, figsize=(12, 6))

# 地图图
axs[0].imshow(map_img)
axs[0].set_title("路径1 地图图", fontsize=14)
axs[0].axis("off")

# 抽象图
pts = abs_data["points"]
lbl = abs_data["labels"]
x, y = zip(*pts)
axs[1].set_xlim(0, yellow_img.shape[1])
```

```python
axs[1].set_ylim(yellow_img.shape[0], 0)
axs[1].set_aspect('equal')
axs[1].set_title("路径1 抽象图", fontsize=14)
axs[1].grid(True, linestyle="--", alpha=0.4)

# 点和编号
for j in range(len(pts)):
    axs[1].plot(x[j], y[j], 'ro', markersize=8)
    axs[1].text(x[j] + 10, y[j] - 10, str(lbl[j]), fontsize=12, color='blue', weight='bold')

# 箭头连接
for j in range(len(pts) - 1):
    x0, y0 = pts[j]
    x1, y1 = pts[j+1]
    axs[1].arrow(x0, y0, x1 - x0, y1 - y0,
                 head_width=10, head_length=15,
                 fc='green', ec='green', length_includes_head=True)

plt.tight_layout()
plt.show()
```